Exp No. 1	Data Format Conversion in IoT Systems
Date:	

Aim:

To acquire temperature sensor data using an Arduino Uno in Tinkercad, download the generated data, and convert the data into the required format using MATLAB.

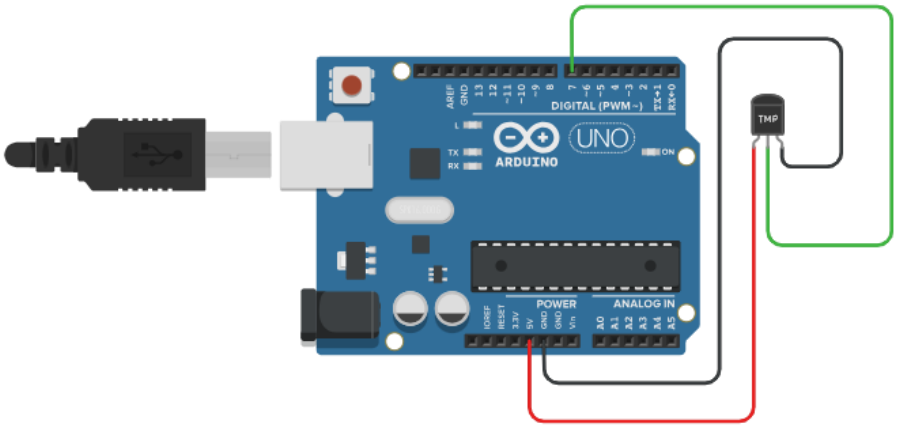
Components Required:

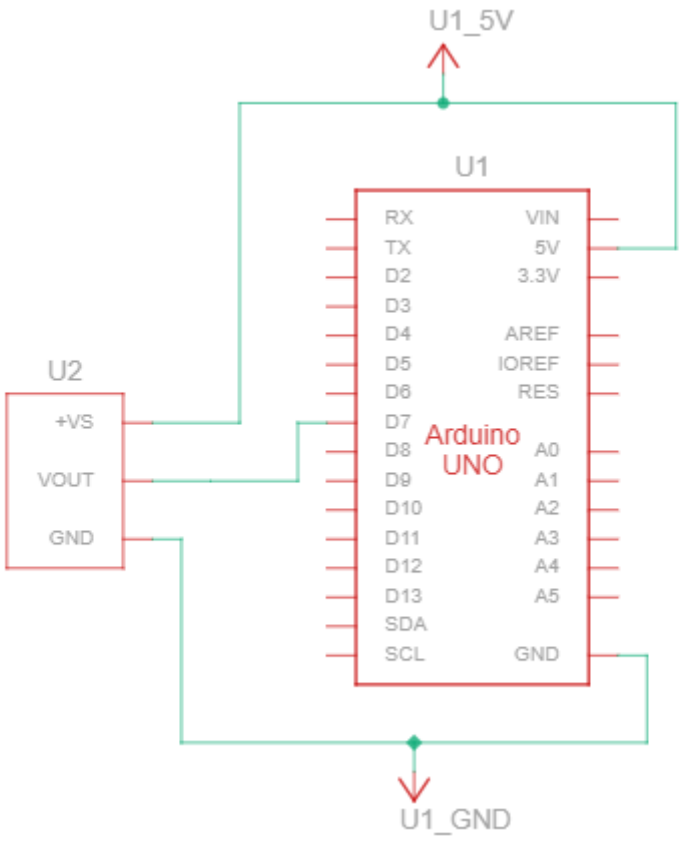
- Arduino Uno
- TMP36 (or LM35) Temperature Sensor
- Jumper wires
- USB cable
- MATLAB software

Program:

```
const int tempPin = A0;
void setup() {
  Serial.begin(9600);
  Serial.println("Time(s),Temperature(C)");
}
void loop() {
  int sensorValue = analogRead(tempPin);
  float voltage = sensorValue * (5.0 / 1023.0);
  float tempC = (voltage - 0.5) * 100.0;
  Serial.print(millis() / 1000.0);
  Serial.print(",");
  Serial.println(tempC);
  delay(1000);
}
```

Diagram

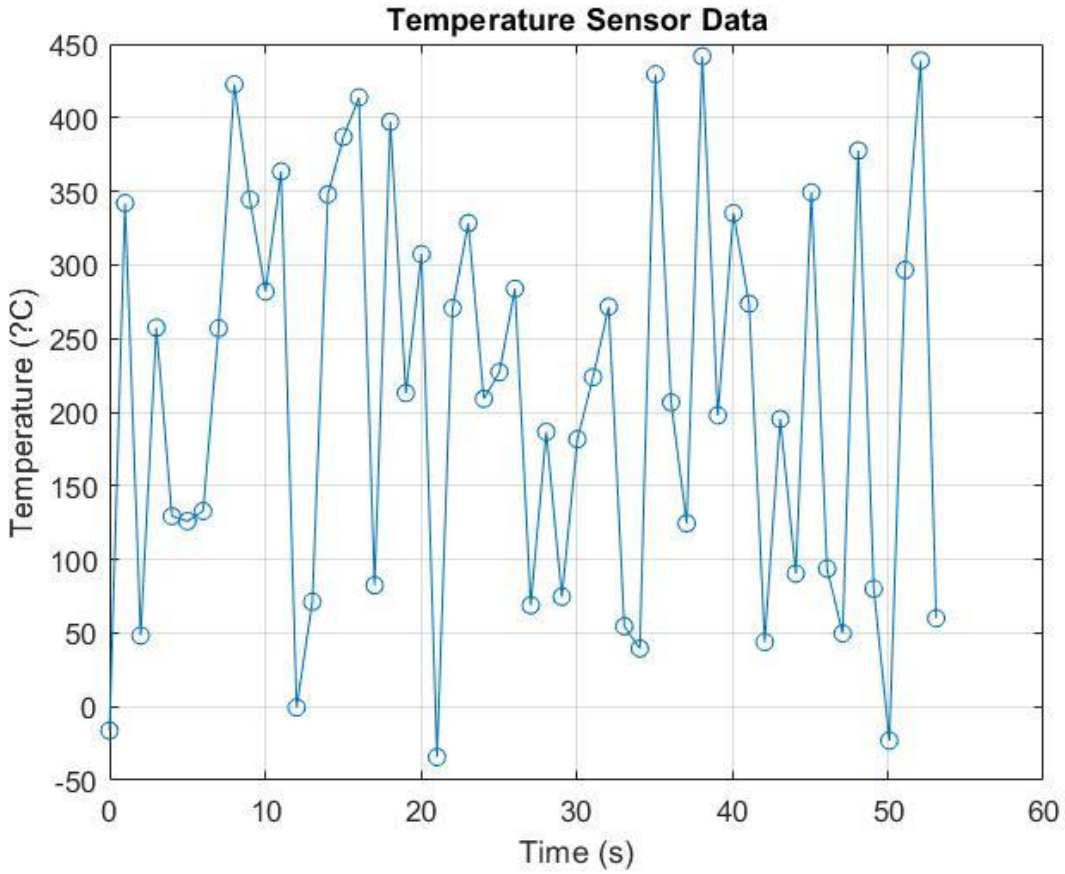

DIGITAL (PWM~)
UNO
ARDUINO
POWER
ANALOG IN
TMP


U1_5V
U1
RX
TX
D2
D3
D4
D5
D6
D7
D8
D9
D10
D11
D12
D13
SDA
SCL
VIN
5V
3.3V
AREF
IOREF
RES
A0
A1
A2
A3
A4
A5
GND
Arduino
UNO
U2
+VS
VOUT
GND
U1_GND

MATLAB Program

```
clc;
clear all;
close all;
data = csvread('temperature_data.csv',1,0);
time = data(:,1);
temp = data(:,2);
disp('Imported Temperature Data:');
disp(data);
figure;
plot(time,temp,'-o','LineWidth',1.5);
xlabel('Time (seconds)');
ylabel('Temperature (Degree C)');
title('Temperature Sensor Data from Tinkercad');
grid on;
save('temperatureData.mat','data');
xlswrite('temperatureData.xlsx',data);
csvwrite('converted_temperature_data.csv',data);
disp('---------------------------------');
disp('Data Format Conversion Completed');
disp('MAT file created successfully');
disp('Excel file created successfully');
disp('CSV conversion completed successfully');
disp('---------------------------------');
```

Output


Temperature Sensor Data
Temperature (?C)
Time (s)
450
400
350
300
250
200
150
100
50
0
-50
0
10
20
30
40
50
60

Result:

The temperature data was successfully acquired from the TMP36 sensor using the Arduino Uno in Tinkercad. The serial output was downloaded as a CSV file and imported into MATLAB. MATLAB successfully converted the data into different formats (MAT and Excel) and displayed the data graphically. Hence, the data format conversion in the IoT system was carried out successfully.

Exp No. 2	Simulation of IoT Connectivity Technologies
Date:	

Aim

To generate sensor data using an Arduino-based Tinkercad circuit and simulate its transmission through different IoT communication technologies using MATLAB.

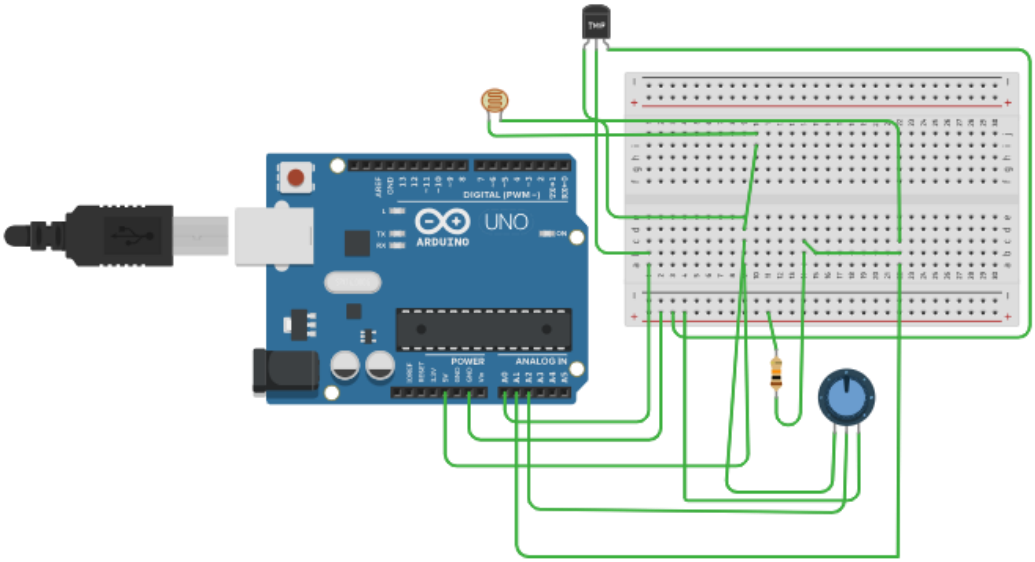
Components Required

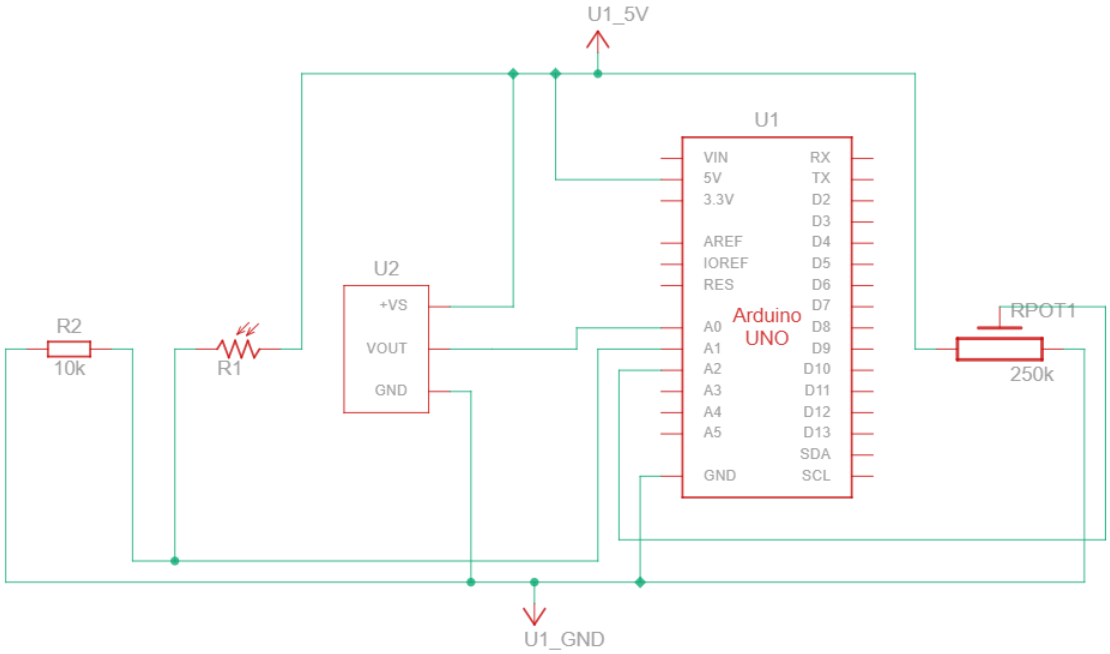
- Arduino Uno
- Breadboard
- LDR Sensor
- Potentiometer
- TMP36 (or LM35) Temperature Sensor
- Jumper wires
- USB cable
- MATLAB software

Program

```
const int tempPin = A0;
const int lightPin = A1;
const int soilPin = A2;
void setup()
{
  Serial.begin(9600);
}
void loop()
{
  int tempValue = analogRead(tempPin);
  float voltage = tempValue * 5.0 / 1023.0;
  float temperature = (voltage - 0.5) * 100;
  int light = analogRead(lightPin);
```

Diagram


UNO
ARDUINO
DIGITAL (PWM~)
POWER
ANALOG IN


U1_5V
U1
VIN
5V
3.3V
AREF
IOREF
RES
A0
A1
A2
A3
A4
A5
GND
RX
TX
D2
D3
D4
D5
D6
D7
D8
D9
D10
D11
D12
D13
SDA
SCL
Arduino
UNO
U2
+VS
VOUT
GND
R2
10k
R1
RPOT1
250k
U1_GND

```
  int soil = analogRead(soilPin);
  Serial.print(temperature);
  Serial.print(",");
  Serial.print(light);
  Serial.print(",");
  Serial.println(soil);
  delay(1000);
}
```

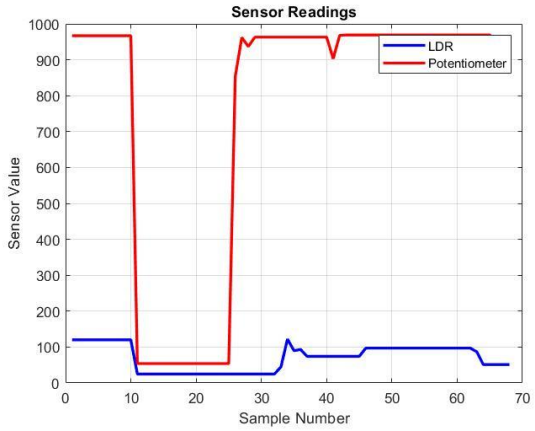
MATLAB Program

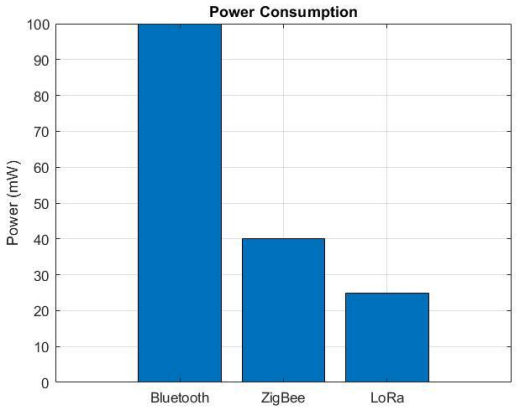
```
clc; clear; close all;
data = csvread('SensorData.csv');
LDR = data(:,1);
POT = data(:,2);
Samples = 1:length(LDR);
figure;
plot(Samples,LDR,'b','LineWidth',2);
hold on;
plot(Samples,POT,'r','LineWidth',2);
grid on;
xlabel('Sample Number');
ylabel('Sensor Value');
title('Sensor Readings');
legend('LDR','Potentiometer');
Protocols = {'Bluetooth','ZigBee','LoRa'};
Range = [10 100 15000];          % meters
DataRate = [2 0.25 0.05];        % Mbps
Power = [100 40 25];             % mW
Latency = [10 35 300];           % ms
PDR = [98 98 96];                % %
```

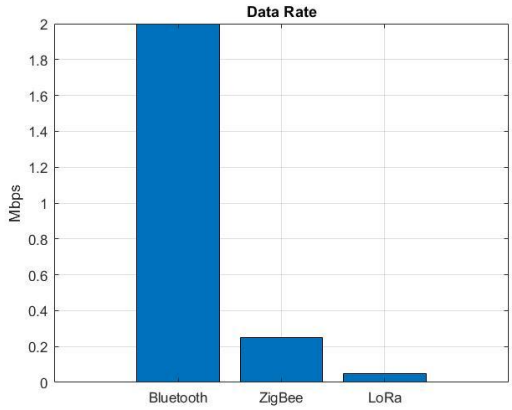
```
fprintf('\n');
disp('-----------------------------------------------');
disp('      IoT Protocol Simulation');
disp('-----------------------------------------------');
for i = 1:length(Protocols)
    fprintf('\nProtocol : %s\n',Protocols{i});
    fprintf('Range               : %.2f m\n',Range(i));
    fprintf('Data Rate           : %.2f Mbps\n',DataRate(i));
    fprintf('Power Consumption   : %.2f mW\n',Power(i));
    fprintf('Latency             : %.2f ms\n',Latency(i));
    fprintf('Packet Delivery     : %.2f %%\n',PDR(i));
end
figure;
bar(Range);
grid on;
title('Communication Range');
ylabel('Range (m)');
set(gca,'XTickLabel',Protocols);
figure;
bar(DataRate);
grid on;
title('Data Rate');
ylabel('Mbps');
set(gca,'XTickLabel',Protocols);
figure;
bar(Power);
grid on;
title('Power Consumption');
ylabel('Power (mW)');
```

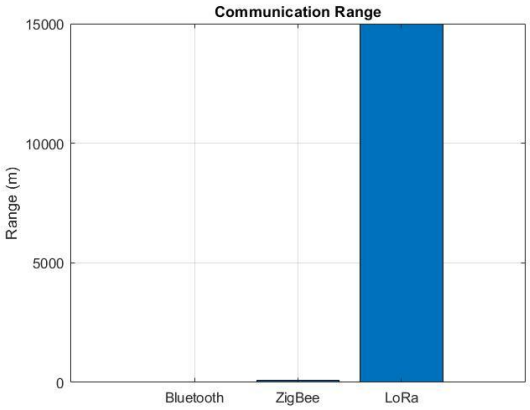
```
set(gca,'XTickLabel',Protocols);
figure;
bar(Latency);
grid on;
title('Latency');
ylabel('Milliseconds');
set(gca,'XTickLabel',Protocols);
figure;
bar(PDR);
grid on;
title('Packet Delivery Ratio');
ylabel('PDR (%)');
ylim([90 100]);
set(gca,'XTickLabel',Protocols);
disp(' ');
disp('----------------------------------------');
disp('Packet Transmission');
disp('----------------------------------------');
for i = 1:length(LDR)
    fprintf('\nPacket %d\n',i);
    fprintf('LDR Value          : %d\n',LDR(i));
    fprintf('Potentiometer Value : %d\n',POT(i));
    fprintf('\n');
    fprintf('Bluetooth  : Packet Sent Successfully\n');
    pause(0.1);
    fprintf('ZigBee     : Packet Sent Successfully\n');
    pause(0.1);
    fprintf('LoRa       : Packet Sent Successfully\n');
    pause(0.1);
```

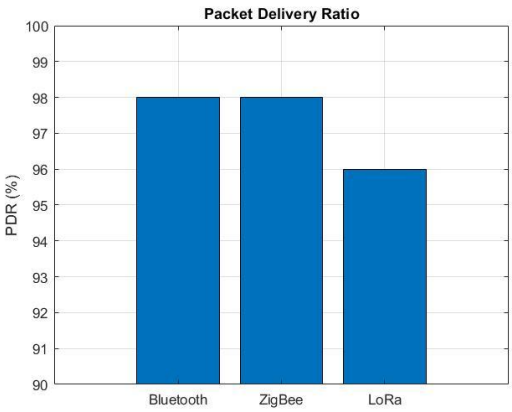
Output

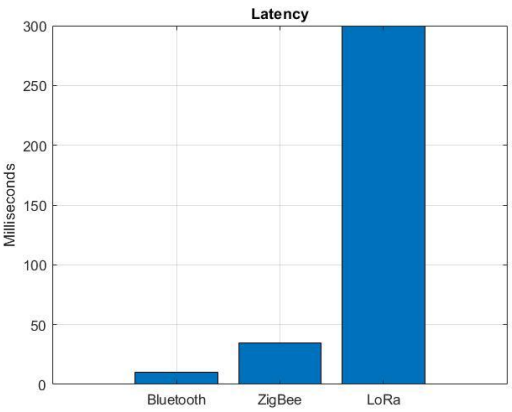

Sensor Readings
LDR
Potentiometer
Sensor Value
Sample Number
0
100
200
300
400
500
600
700
800
900
1000
0
10
20
30
40
50
60
70


Power Consumption
Power (mW)
0
10
20
30
40
50
60
70
80
90
100
Bluetooth
ZigBee
LoRa


Data Rate
Mbps
0
0.2
0.4
0.6
0.8
1
1.2
1.4
1.6
1.8
2
Bluetooth
ZigBee
LoRa


Communication Range
Range (m)
0
5000
10000
15000
Bluetooth
ZigBee
LoRa


Packet Delivery Ratio
PDR (%)
90
91
92
93
94
95
96
97
98
99
100
Bluetooth
ZigBee
LoRa


Latency
Milliseconds
0
50
100
150
200
250
300
Bluetooth
ZigBee
LoRa

```
end
[~,r]=max(Range);
[~,d]=max(DataRate);
[~,p]=min(Power);
disp(' ');
disp('----------------------------------------');
fprintf('Best Long Range Protocol : %s\n',Protocols{r});
fprintf('Highest Data Rate        : %s\n',Protocols{d});
fprintf('Lowest Power Protocol    : %s\n',Protocols{p});
disp('----------------------------------------');
disp('Simulation Completed Successfully');
```

Result

The sensor data was successfully generated using Arduino in Tinkercad and imported into MATLAB. Different IoT communication technologies were simulated and compared based on transmission range, data rate, and power consumption. The experiment demonstrated that the choice of communication protocol depends on the application requirements.

Exp No. 3	Edge Processing Implementation for IoT Data
Date:	

Aim:

To implement edge processing techniques on IoT sensor data using MATLAB by performing local data filtering, threshold-based decision making, anomaly detection, and transmission optimization.

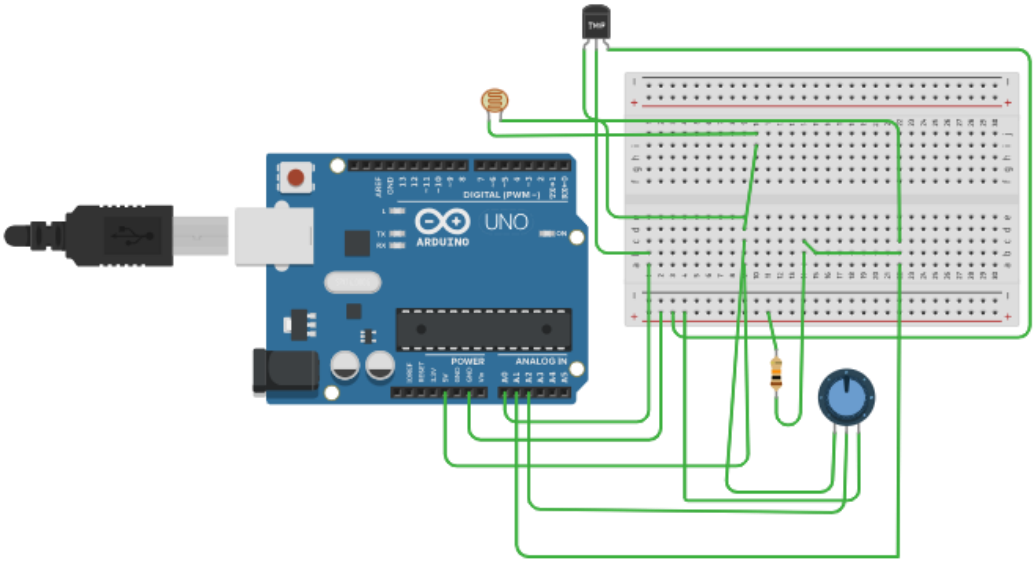
Components Required:

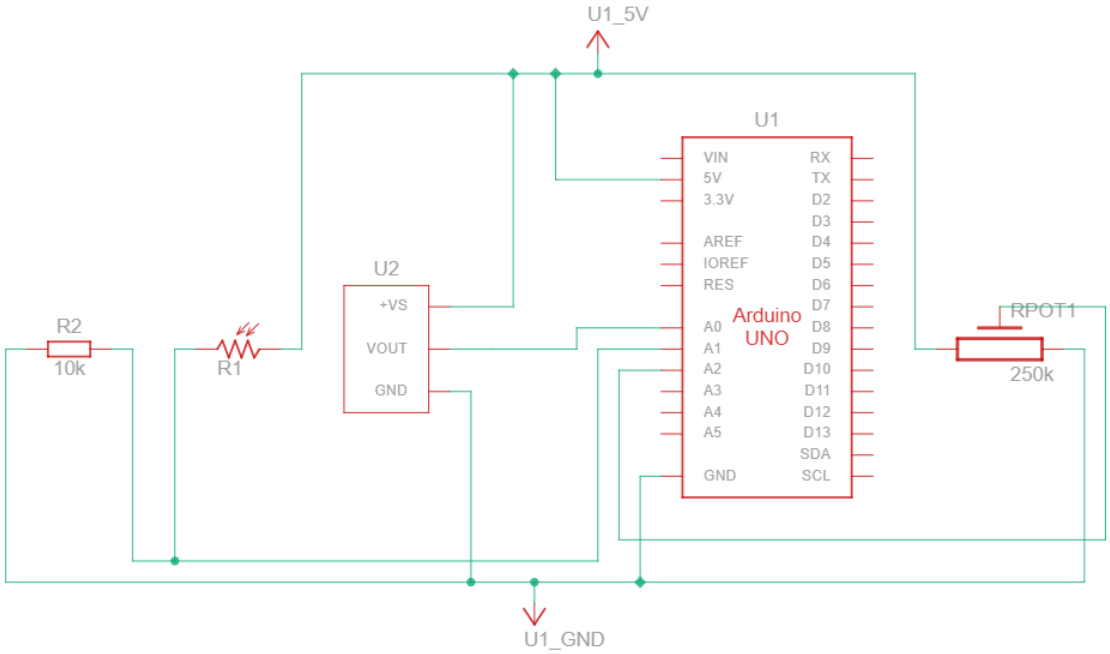
- Arduino Uno
- Breadboard
- LDR Sensor
- Potentiometer
- TMP36 (or LM35) Temperature Sensor
- Jumper wires
- USB cable
- MATLAB software

Program

```
const int tempPin = A0;
const int lightPin = A1;
const int soilPin = A2;
void setup()
{
  Serial.begin(9600);
}
void loop()
{
  int tempValue = analogRead(tempPin);
  float voltage = tempValue * 5.0 / 1023.0;
  float temperature = (voltage - 0.5) * 100;
  int light = analogRead(lightPin);
```

Diagram


UNO
ARDUINO
DIGITAL (PWM~)
POWER
ANALOG IN


U1_5V
U1
VIN
5V
3.3V
AREF
IOREF
RES
A0
A1
A2
A3
A4
A5
GND
RX
TX
D2
D3
D4
D5
D6
D7
D8
D9
D10
D11
D12
D13
SDA
SCL
Arduino
UNO
U2
+VS
VOUT
GND
R2
10k
R1
RPOT1
250k
U1_GND

```
    int soil = analogRead(soilPin);
    Serial.print(temperature);
    Serial.print(",");
    Serial.print(light);
    Serial.print(",");
    Serial.println(soil);
    delay(1000);
  }
```

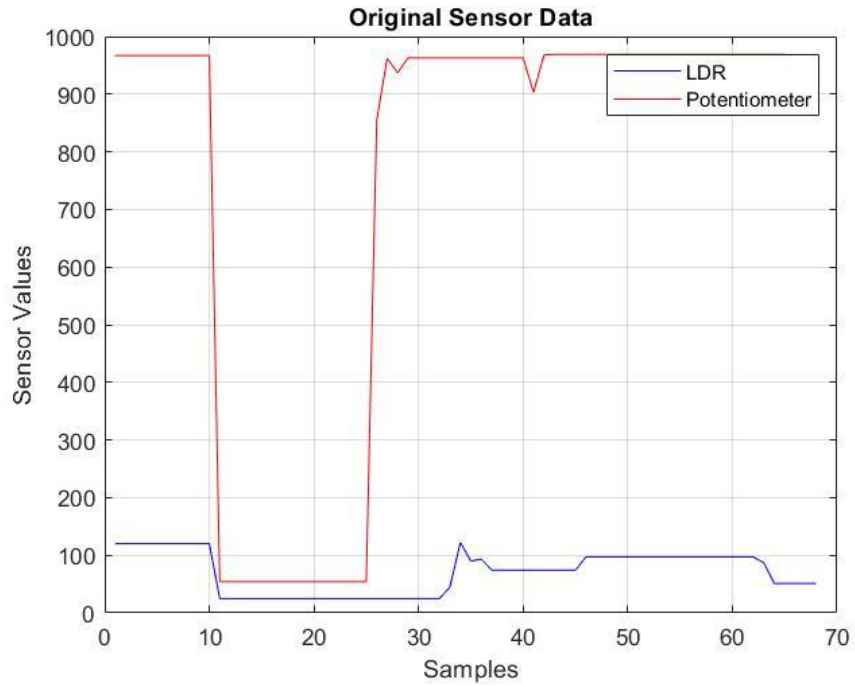
Matlab Program

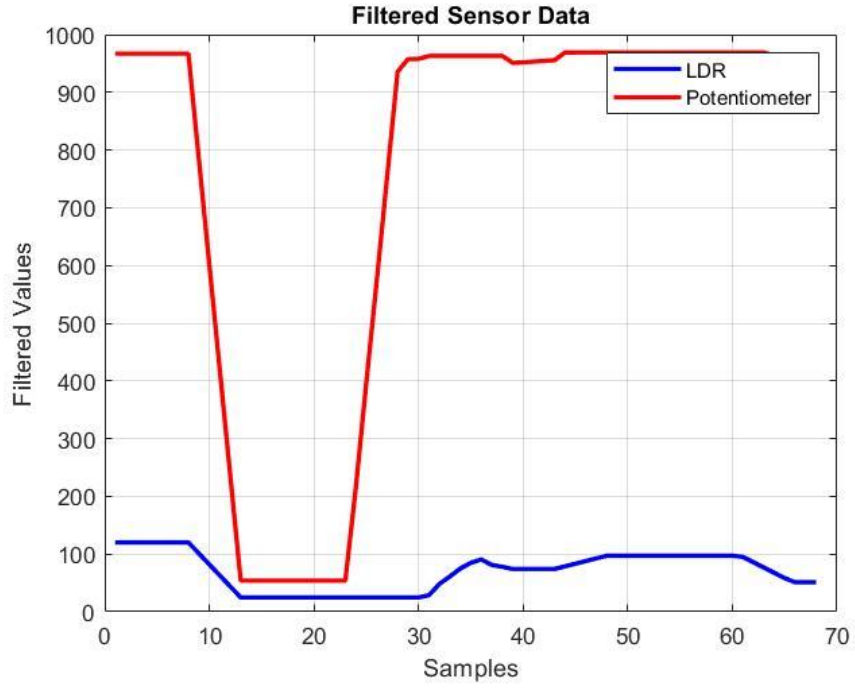
```
clc;
clear;
close all;
%% Read Sensor Data
data = csvread('SensorData.csv');
LDR = data(:,1);
POT = data(:,2);
Samples = 1:length(LDR);
%% Plot Original Sensor Data
figure;
plot(Samples,LDR,'b');
hold on;
plot(Samples,POT,'r');
grid on;
title('Original Sensor Data');
xlabel('Samples');
ylabel('Sensor Values');
legend('LDR','Potentiometer');
%% Edge Processing
% Moving Average Filter
```

```
FilteredLDR = movmean(LDR,5);
FilteredPOT = movmean(POT,5);
figure;
plot(Samples,FilteredLDR,'b','LineWidth',2);
hold on;
plot(Samples,FilteredPOT,'r','LineWidth',2);
grid on;
title('Filtered Sensor Data');
xlabel('Samples');
ylabel('Filtered Values');
legend('LDR','Potentiometer');
%% Threshold Detection
LDRthreshold = 600;
POTthreshold = 350;
disp(' ');
disp('Threshold Detection');
for i=1:length(LDR)
    if LDR(i)>LDRthreshold
        fprintf('Sample %d : High Light Detected\n',i);
    end
    if POT(i)>POTthreshold
        fprintf('Sample %d : Potentiometer Threshold Exceeded\n',i);
    end
end
%% Anomaly Detection
disp(' ');
disp('Anomaly Detection');
MeanLDR = mean(LDR);
StdLDR = std(LDR);
```

```
for i=1:length(LDR)
    if abs(LDR(i)-MeanLDR)>2*StdLDR
        fprintf('Abnormal LDR Reading at Sample %d\n',i);
    end
end
%% Data Reduction
disp(' ');
disp('Edge Transmission');
Previous = LDR(1);
for i=2:length(LDR)
    if abs(LDR(i)-Previous)>20
        fprintf('Transmit Sample %d : %d\n',i,LDR(i));
        Previous=LDR(i);
    end
end
%% Compression Ratio
OriginalPackets = length(LDR);
ReducedPackets = 0;
Previous = LDR(1);
for i=2:length(LDR)
    if abs(LDR(i)-Previous)>20
        ReducedPackets = ReducedPackets + 1;
        Previous=LDR(i);
    end
end
fprintf('\n');
fprintf('Original Packets : %d\n',OriginalPackets);
fprintf('Packets Sent     : %d\n',ReducedPackets);
fprintf('Reduction        : %.2f %%\n', ...
```

Output


Original Sensor Data
LDR
Potentiometer
Sensor Values
Samples
0
100
200
300
400
500
600
700
800
900
1000
0
10
20
30
40
50
60
70


Filtered Sensor Data
LDR
Potentiometer
Filtered Values
Samples
0
100
200
300
400
500
600
700
800
900
1000
0
10
20
30
40
50
60
70

```
100*(OriginalPackets-ReducedPackets)/OriginalPackets);
```

Result:

The IoT sensor data was successfully processed at the edge using MATLAB. Noise filtering, threshold detection, anomaly detection, and data reduction were performed successfully, demonstrating efficient edge processing before data transmission.

Exp No. 4	Connectivity Quality Validation for IoT Devices
Date:	

Aim:

To simulate and evaluate the connectivity quality of IoT devices by analyzing communication performance metrics such as RSSI, Packet Delivery Ratio, Packet Loss, and Communication Delay using MATLAB.

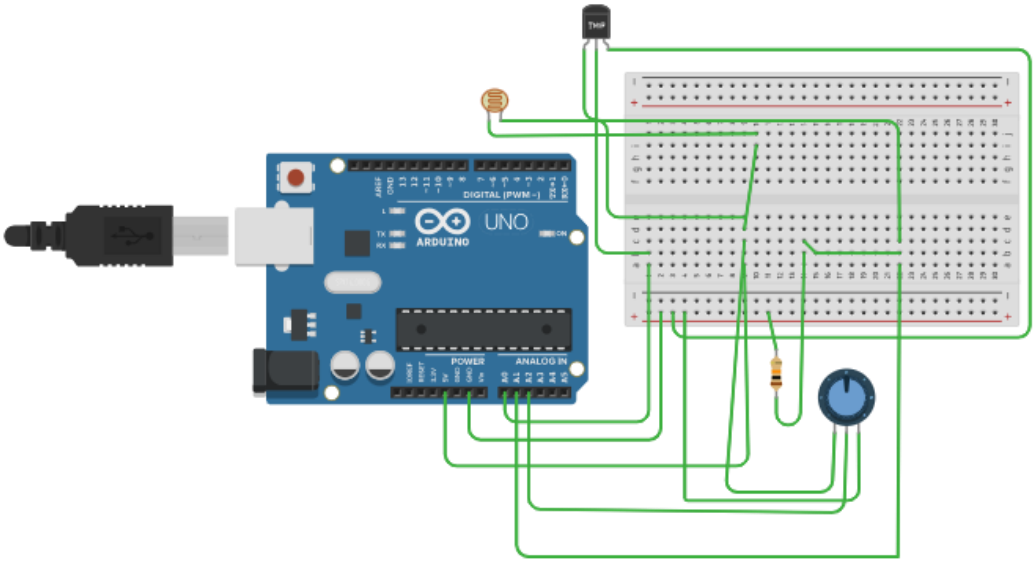
Components Required:

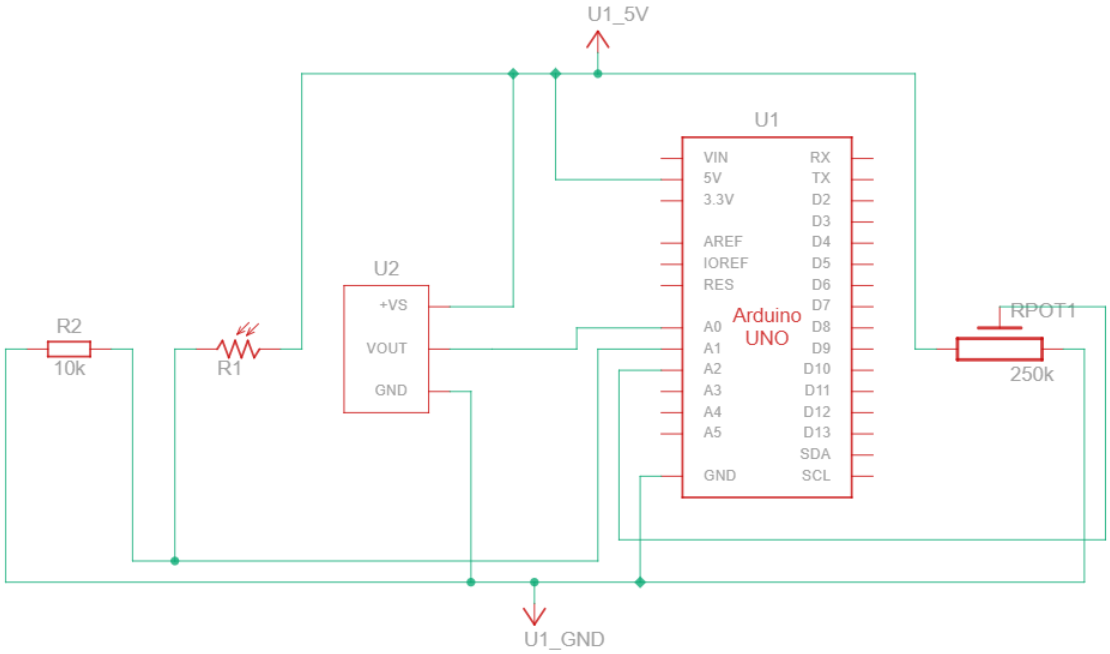
- Arduino Uno
- Breadboard
- LDR Sensor
- Potentiometer
- TMP36 (or LM35) Temperature Sensor
- Jumper wires
- USB cable
- MATLAB software

Program

```
const int tempPin = A0;
const int lightPin = A1;
const int soilPin = A2;
void setup()
{
  Serial.begin(9600);
}
void loop()
{
  int tempValue = analogRead(tempPin);
  float voltage = tempValue * 5.0 / 1023.0;
  float temperature = (voltage - 0.5) * 100;
  int light = analogRead(lightPin);
```

Diagram


UNO
ARDUINO
DIGITAL (PWM~)
POWER
ANALOG IN


U1_5V
U1
VIN
5V
3.3V
AREF
IOREF
RES
A0
A1
A2
A3
A4
A5
GND
RX
TX
D2
D3
D4
D5
D6
D7
D8
D9
D10
D11
D12
D13
SDA
SCL
Arduino
UNO
U2
+VS
VOUT
GND
R2
10k
R1
RPOT1
250k
U1_GND

```
    int soil = analogRead(soilPin);
    Serial.print(temperature);
    Serial.print(",");
    Serial.print(light);
    Serial.print(",");
    Serial.println(soil);
    delay(1000);
  }
```

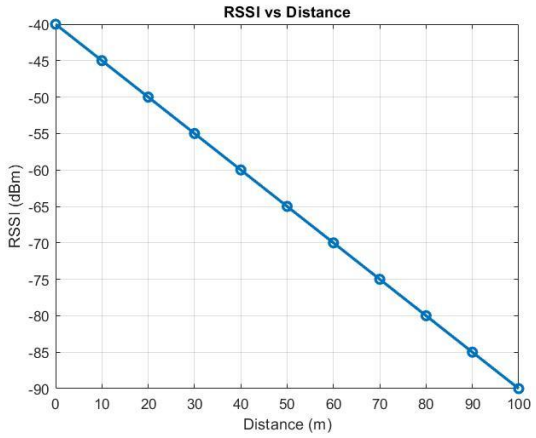
Matlab Program

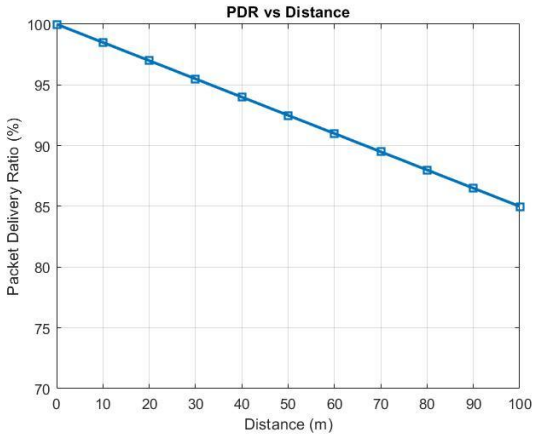
```
clc;
clear;
close all;
D = 0:10:100;
RSSI = -40-0.5*D;
PDR = 100-0.15*D;
Loss = 100-PDR;
Delay = 10+0.25*D;
fprintf('Distance\tRSSI(dBm)\tPDR(%%)\tLoss(%%)\tDelay(ms)\n');
for i = 1:length(D)
   fprintf('%5d\t\t%6.1f\t\t%6.1f\t%6.1f\t%6.1f\n', ...
      D(i),RSSI(i),PDR(i),Loss(i),Delay(i));
end
figure;
plot(D,RSSI,'-o','LineWidth',2);
grid on;
xlabel('Distance (m)');
ylabel('RSSI (dBm)');
title('RSSI vs Distance');
```

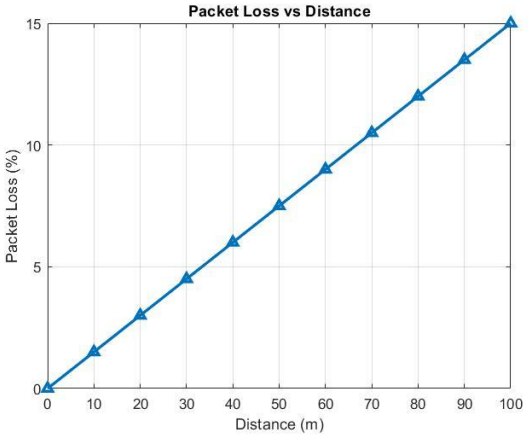
```
figure;
plot(D,PDR,'-s','LineWidth',2);
grid on;
xlabel('Distance (m)');
ylabel('Packet Delivery Ratio (%)');
title('PDR vs Distance');
ylim([70 100]);
figure;
plot(D,Loss,'-^','LineWidth',2);
grid on;
xlabel('Distance (m)');
ylabel('Packet Loss (%)');
title('Packet Loss vs Distance');
figure;
plot(D,Delay,'-d','LineWidth',2);
grid on;
xlabel('Distance (m)');
ylabel('Delay (ms)');
title('Communication Delay vs Distance');
fprintf('\nConnectivity Status\n');
for i = 1:length(D)
    if RSSI(i) > -60
        Status = 'Excellent';
    elseif RSSI(i) > -75
        Status = 'Good';
    elseif RSSI(i) > -90
        Status = 'Weak';
    else
        Status = 'Disconnected';
```

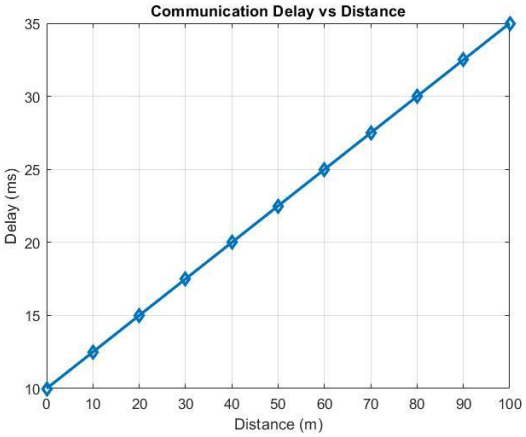
```
    end
    fprintf('Distance = %3d m   RSSI = %5.1f dBm   %s\n', ...
        D(i),RSSI(i),Status);
end
Duser = input('\nEnter Communication Distance (m): ');
R = -40-0.5*Duser;
P = 100-0.15*Duser;
L = 100-P;
T = 10+0.25*Duser;
fprintf('\nConnectivity Report\n');
fprintf('Distance           : %.1f m\n',Duser);
fprintf('RSSI               : %.2f dBm\n',R);
fprintf('PDR                : %.2f %%\n',P);
fprintf('Packet Loss        : %.2f %%\n',L);
fprintf('Communication Delay : %.2f ms\n',T);
if R > -60
    disp('Link Quality : Excellent');
elseif R > -75
    disp('Link Quality : Good');
elseif R > -90
    disp('Link Quality : Weak');
else
    disp('Link Quality : Connection Lost');
end
```

Output:


RSSI vs Distance
RSSI (dBm)
-40
-45
-50
-55
-60
-65
-70
-75
-80
-85
-90
0
10
20
30
40
50
60
70
80
90
100
Distance (m)


PDR vs Distance
Packet Delivery Ratio (%)
100
95
90
85
80
75
70
0
10
20
30
40
50
60
70
80
90
100
Distance (m)


Packet Loss vs Distance
Packet Loss (%)
15
10
5
0
0
10
20
30
40
50
60
70
80
90
100
Distance (m)


Communication Delay vs Distance
Delay (ms)
35
30
25
20
15
10
0
10
20
30
40
50
60
70
80
90
100
Distance (m)

Result:

The connectivity quality of the IoT communication link was simulated using MATLAB by analyzing RSSI, Packet Delivery Ratio, Packet Loss, and Communication Delay at different transmission distances.

Exp No. 5	IoT Device Data Security Implementation
Date:	

Aim

To implement data security techniques for IoT sensor data using MATLAB by performing encryption, decryption, and integrity verification before data transmission.

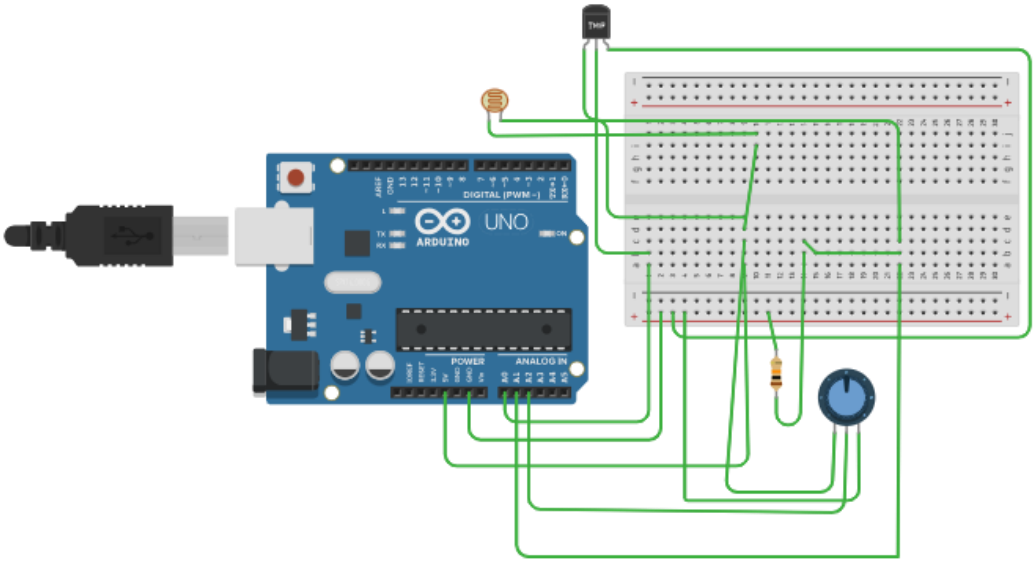
Components Required:

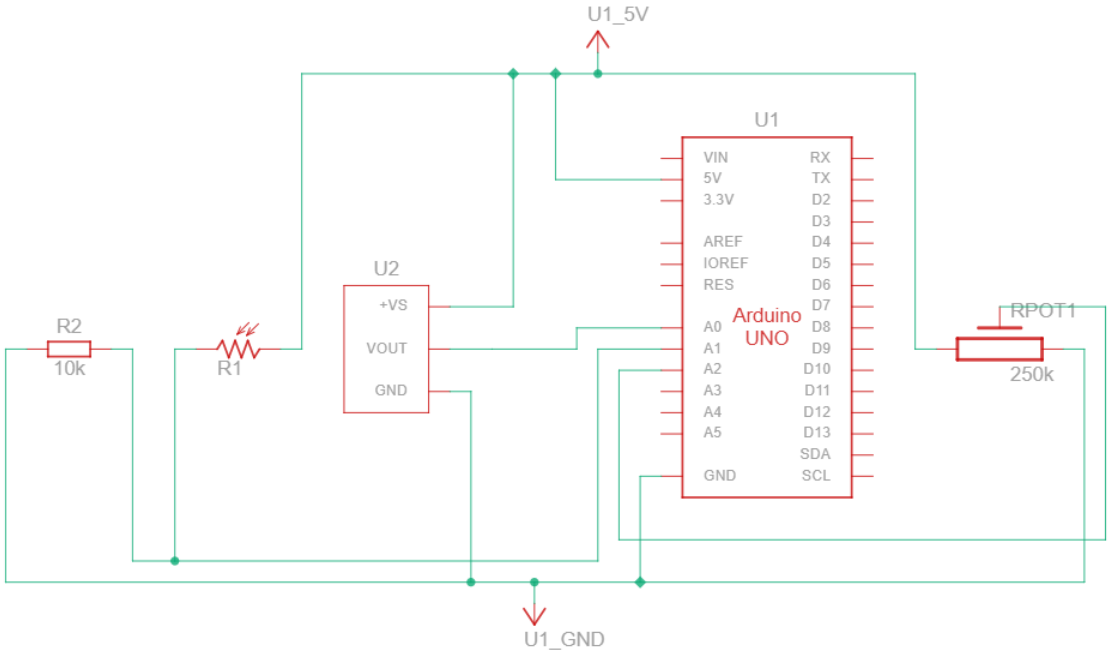
- Arduino Uno
- Breadboard
- LDR Sensor
- Potentiometer
- TMP36 (or LM35) Temperature Sensor
- Jumper wires
- USB cable
- MATLAB software

Program

```
const int tempPin = A0;
const int lightPin = A1;
const int soilPin = A2;
void setup()
{
  Serial.begin(9600);
}
void loop()
{
  int tempValue = analogRead(tempPin);
  float voltage = tempValue * 5.0 / 1023.0;
  float temperature = (voltage - 0.5) * 100;
  int light = analogRead(lightPin);
```

Diagram


UNO
ARDUINO
DIGITAL (PWM~)
POWER
ANALOG IN


U1_5V
U1
VIN
5V
3.3V
AREF
IOREF
RES
A0
A1
A2
A3
A4
A5
GND
RX
TX
D2
D3
D4
D5
D6
D7
D8
D9
D10
D11
D12
D13
SDA
SCL
Arduino
UNO
U2
+VS
VOUT
GND
R2
10k
R1
RPOT1
250k
U1_GND

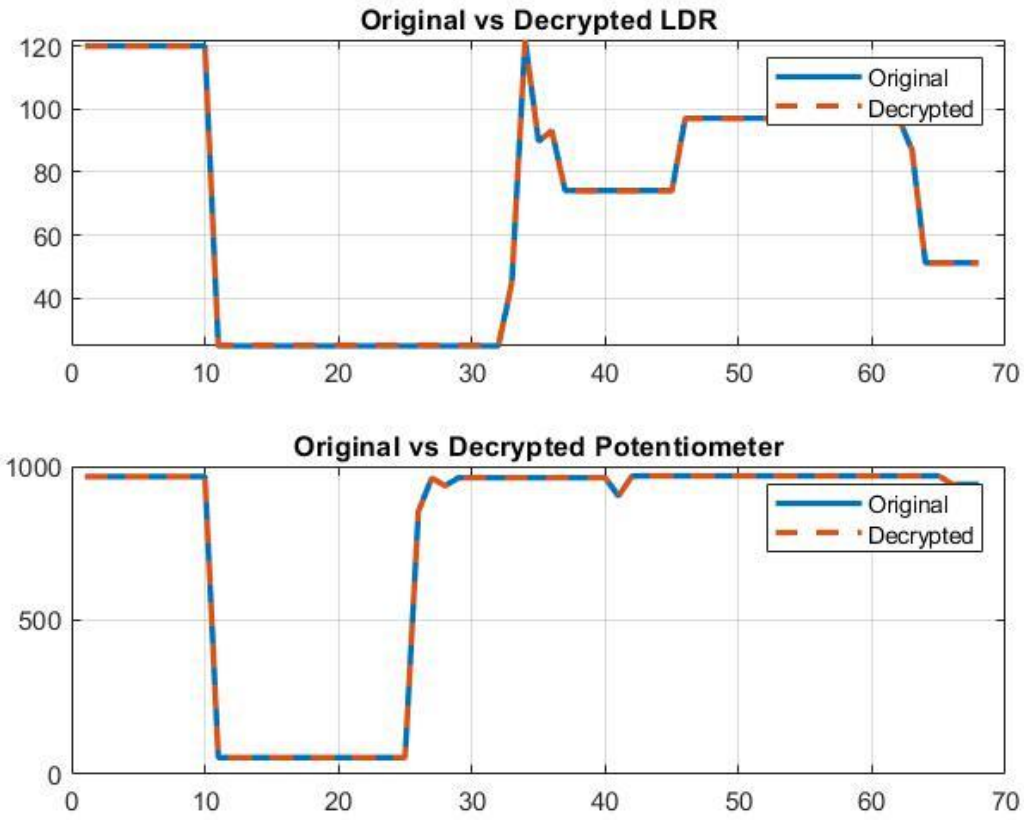
```
    int soil = analogRead(soilPin);
    Serial.print(temperature);
    Serial.print(",");
    Serial.print(light);
    Serial.print(",");
    Serial.println(soil);
    delay(1000);
  }
```

Matlab Program

```
clc;
clear;
close all;
%% Read Sensor Data
data = csvread('SensorData.csv');
LDR = data(:,1);
POT = data(:,2);
%% Display Original Data
disp('Original Sensor Data');
disp(table(LDR,POT));
%% Encryption Key
Key = 85;      % Decimal key
%% Encrypt Sensor Data
EncryptedLDR = bitxor(uint16(LDR),Key);
EncryptedPOT = bitxor(uint16(POT),Key);
disp(' ');
disp('Encrypted Data');
disp(table(EncryptedLDR,EncryptedPOT));
%% Decrypt Data
DecryptedLDR = bitxor(EncryptedLDR,Key);
```

```
DecryptedPOT = bitxor(EncryptedPOT,Key);
disp(' ');
disp('Decrypted Data');
disp(table(double(DecryptedLDR),double(DecryptedPOT)));
%% Integrity Verification
if isequal(LDR,double(DecryptedLDR)) && ...
   isequal(POT,double(DecryptedPOT))
    disp(' ');
    disp('Integrity Verification : SUCCESS');
else
    disp(' ');
    disp('Integrity Verification : FAILED');
end
%% Visualization
figure;
subplot(2,1,1)
plot(LDR,'LineWidth',2)
hold on
plot(double(DecryptedLDR),'--','LineWidth',2)
grid on
title('Original vs Decrypted LDR')
legend('Original','Decrypted')
subplot(2,1,2)
plot(POT,'LineWidth',2)
hold on
plot(double(DecryptedPOT),'--','LineWidth',2)
grid on
title('Original vs Decrypted Potentiometer')
legend('Original','Decrypted')
```

Output


Original vs Decrypted LDR
120
100
80
60
40
0
10
20
30
40
50
60
70
Original
Decrypted
Original vs Decrypted Potentiometer
1000
500
0
0
10
20
30
40
50
60
70
Original
Decrypted

Result

The IoT sensor data was successfully secured using a symmetric encryption technique. The encrypted data was accurately decrypted using the same secret key, and the integrity verification confirmed that the original sensor values were recovered without modification, demonstrating secure data transmission for IoT applications.

Exp No. 6	IoT Device Energy Consumption Modeling
Date:	

Aim:

To model the battery lifetime of an IoT device by estimating energy consumption during sensing, transmission, and sleep modes using MATLAB.

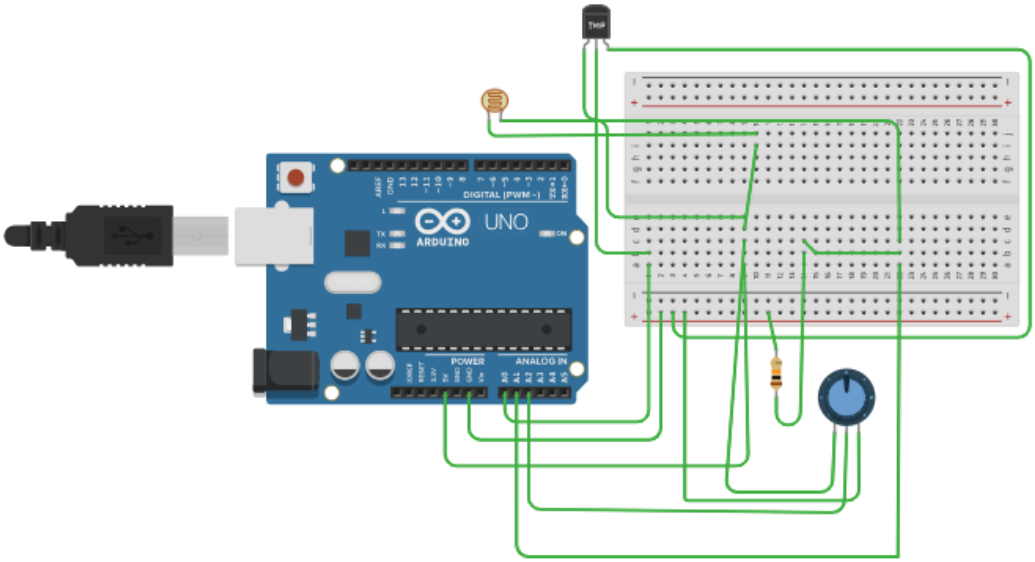
Components Required:

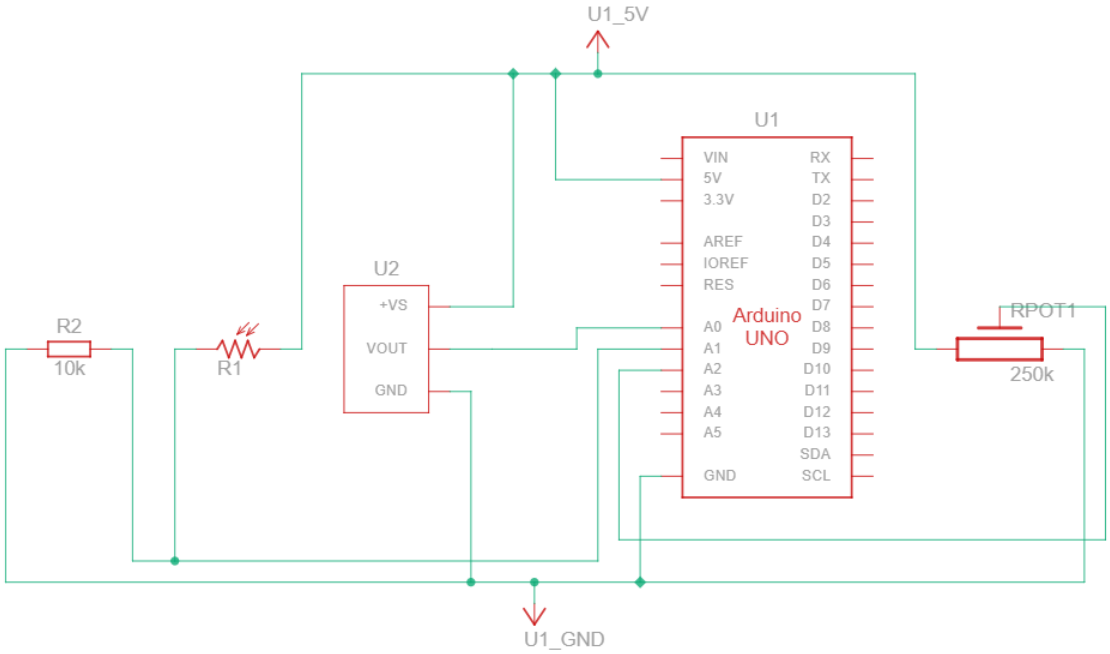
- Arduino Uno
- Breadboard
- LDR Sensor
- Potentiometer
- TMP36 (or LM35) Temperature Sensor
- Jumper wires
- USB cable
- MATLAB software

Program

```
const int tempPin = A0;
const int lightPin = A1;
const int soilPin = A2;
void setup()
{
  Serial.begin(9600);
}
void loop()
{
  int tempValue = analogRead(tempPin);
  float voltage = tempValue * 5.0 / 1023.0;
  float temperature = (voltage - 0.5) * 100;
  int light = analogRead(lightPin);
```

Diagram


UNO
ARDUINO
DIGITAL (PWM~)
POWER
ANALOG IN


U1_5V
U1
VIN
5V
3.3V
AREF
IOREF
RES
A0
A1
A2
A3
A4
A5
GND
RX
TX
D2
D3
D4
D5
D6
D7
D8
D9
D10
D11
D12
D13
SDA
SCL
Arduino
UNO
U2
+VS
VOUT
GND
R2
10k
R1
RPOT1
250k
U1_GND

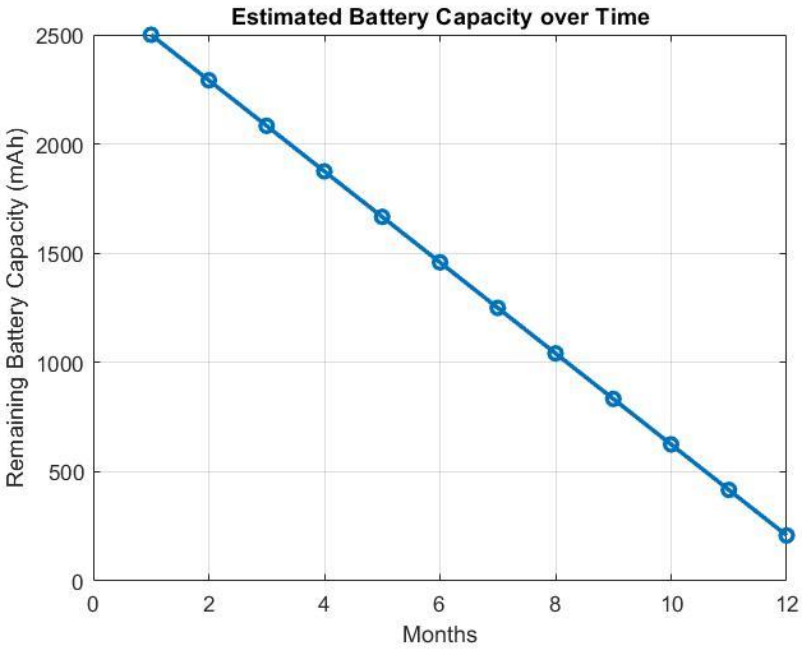
```
    int soil = analogRead(soilPin);
    Serial.print(temperature);
    Serial.print(",");
    Serial.print(light);
    Serial.print(",");
    Serial.println(soil);
    delay(1000);
  }
```

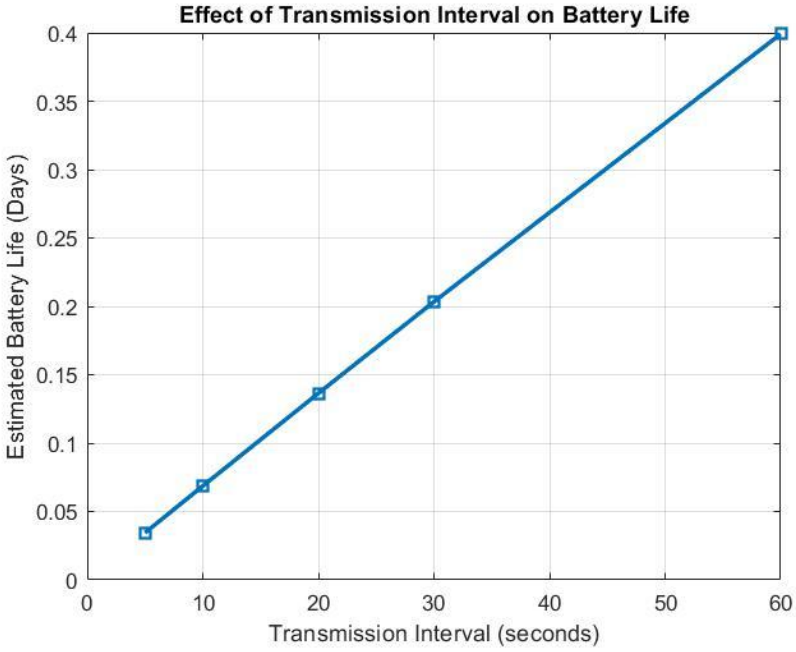
Matlab Program

```
clc;
clear;
close all;
data = csvread('SensorData.csv');
LDR = data(:,1);
POT = data(:,2);
N = length(LDR);
Capacity = 2500;
Voltage = 3.7;
BatteryEnergy = Capacity*Voltage;
SleepCurrent = 0.01;
ActiveCurrent = 15;
TransmitCurrent = 40;
Interval = 10;
ActiveTime = 0.5;
TransmitTime = 0.2;
SleepTime = Interval-ActiveTime-TransmitTime;
DutyCycle = (ActiveTime+TransmitTime)/Interval*100;
ActiveEnergy = ActiveCurrent*ActiveTime;
TransmitEnergy = TransmitCurrent*TransmitTime;
```

```
SleepEnergy = SleepCurrent*SleepTime;
EnergyCycle = ActiveEnergy+TransmitEnergy+SleepEnergy;
CyclesDay = 86400/Interval;
DailyEnergy = EnergyCycle*CyclesDay;
LifeDays = BatteryEnergy/DailyEnergy;
LifeMonths = LifeDays/30;
LifeYears = LifeDays/365;
fprintf('Battery Capacity       : %.0f mAh\n',Capacity);
fprintf('Duty Cycle             : %.2f %%\n',DutyCycle);
fprintf('Daily Energy           : %.2f mWh\n',DailyEnergy);
fprintf('Estimated Life (Days)  : %.2f\n',LifeDays);
fprintf('Estimated Life (Months): %.2f\n',LifeMonths);
fprintf('Estimated Life (Years) : %.2f\n',LifeYears);
Time = 1:12;
RemainingBattery = Capacity-(Time-1)*(Capacity/12);
figure;
plot(Time,RemainingBattery,'-o','LineWidth',2);
grid on;
xlabel('Months');
ylabel('Remaining Battery Capacity (mAh)');
title('Estimated Battery Capacity over Time');
Intervals = [5 10 20 30 60];
Life = zeros(size(Intervals));
for i = 1:length(Intervals)
    T = Intervals(i);
    Sleep = T-ActiveTime-TransmitTime;
    Energy = ActiveCurrent*ActiveTime + ...
             TransmitCurrent*TransmitTime + ...
             SleepCurrent*Sleep;
```

Output


Estimated Battery Capacity over Time
Remaining Battery Capacity (mAh)
Months
0
500
1000
1500
2000
2500
0
2
4
6
8
10
12


Effect of Transmission Interval on Battery Life
Estimated Battery Life (Days)
Transmission Interval (seconds)
0
0.05
0.1
0.15
0.2
0.25
0.3
0.35
0.4
0
10
20
30
40
50
60

```
    Life(i) = BatteryEnergy/(Energy*86400/T);
end
figure;
plot(Intervals,Life,'-s','LineWidth',2);
grid on;
xlabel('Transmission Interval (seconds)');
ylabel('Estimated Battery Life (Days)');
title('Effect of Transmission Interval on Battery Life');
```

Result

The battery lifetime of the IoT device was successfully estimated using MATLAB. The analysis demonstrated that increasing the transmission interval and reducing the duty cycle significantly improve battery life.

Exp No. 7	Battery Life Estimation for IoT Devices
Date:	

Aim

To estimate the battery life of an IoT device using MATLAB by analyzing the effect of transmission interval and duty cycle on battery utilization.

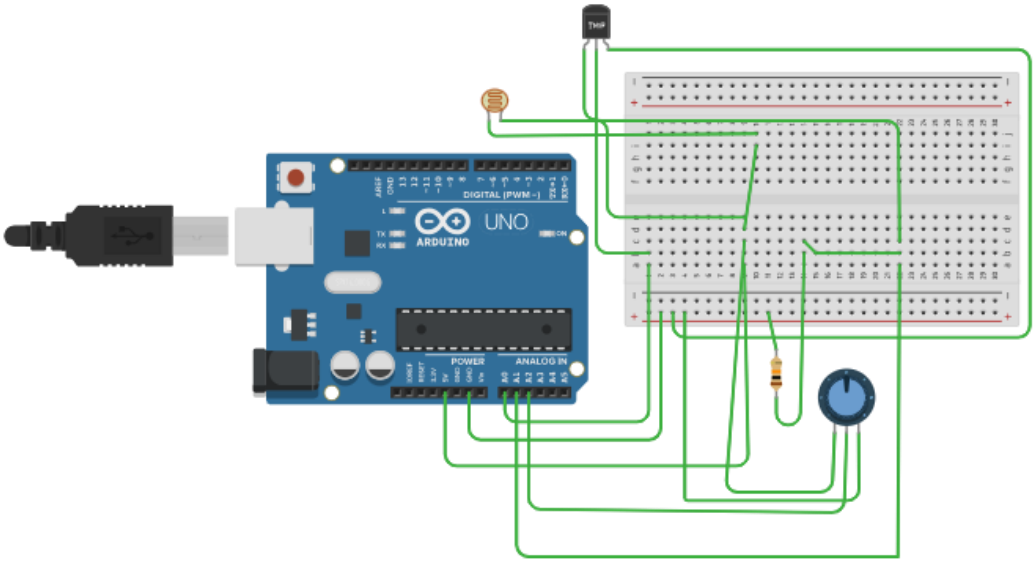
Components Required

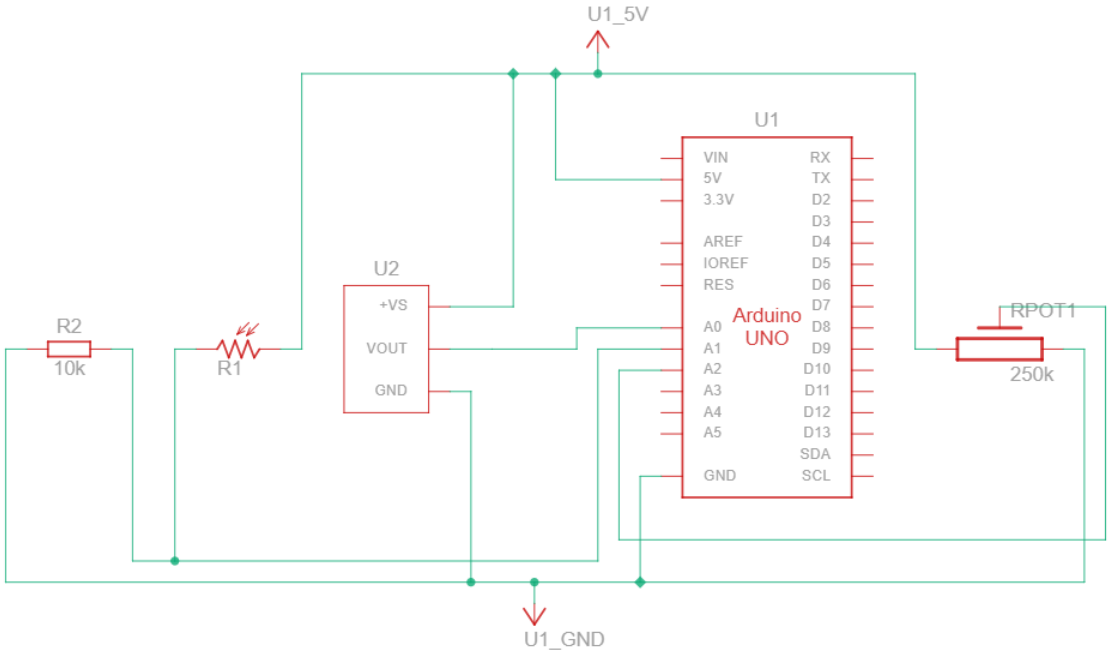
- Arduino UNO
- LDR Sensor
- 10 kΩ Potentiometer
- 10 kΩ Resistor
- Breadboard
- Connecting Wires
- MATLAB
- Tinkercad

Program

```
const int tempPin = A0;
const int lightPin = A1;
 const int soilPin = A2;
 void setup()
 {
   Serial.begin(9600);
 }
 void loop()
 {
   int tempValue = analogRead(tempPin);
   float voltage = tempValue * 5.0 / 1023.0;
   float temperature = (voltage - 0.5) * 100;
   int light = analogRead(lightPin);
```

Diagram


UNO
ARDUINO
DIGITAL (PWM~)
POWER
ANALOG IN


U1_5V
U1
VIN
5V
3.3V
AREF
IOREF
RES
A0
A1
A2
A3
A4
A5
GND
RX
TX
D2
D3
D4
D5
D6
D7
D8
D9
D10
D11
D12
D13
SDA
SCL
Arduino
UNO
U2
+VS
VOUT
GND
R2
10k
R1
RPOT1
250k
U1_GND

```
    int soil = analogRead(soilPin);
    Serial.print(temperature);
    Serial.print(",");
    Serial.print(light);
    Serial.print(",");
    Serial.println(soil);
    delay(1000);
  }
```

MATLAB Program

```
clc;
clear;
close all;
data = csvread('SensorData.csv');
LDR = data(:,1);
POT = data(:,2);
N = length(LDR);
BatteryCapacity = 2500;
ActiveCurrent = 15;
TransmitCurrent = 40;
SleepCurrent = 0.01;
Interval = 10;
ActiveTime = 0.5;
TransmitTime = 0.2;
SleepTime = Interval-ActiveTime-TransmitTime;
TotalTime = N*Interval;
ActiveTotal = N*ActiveTime;
TransmitTotal = N*TransmitTime;
SleepTotal = N*SleepTime;
ActiveCharge = ActiveCurrent*ActiveTotal/3600;
```

```
TransmitCharge = TransmitCurrent*TransmitTotal/3600;
SleepCharge = SleepCurrent*SleepTotal/3600;
TotalCharge = ActiveCharge+TransmitCharge+SleepCharge;
AverageCurrent = TotalCharge/(TotalTime/3600);
LifeHours = BatteryCapacity/AverageCurrent;
LifeDays = LifeHours/24;
LifeMonths = LifeDays/30;
BatteryUsed = TotalCharge/BatteryCapacity*100;
fprintf('Number of Sensor Samples       : %d\n',N);
fprintf('Number of Sampling Events      : %d\n',N);
fprintf('Number of Transmission Events : %d\n',N);
fprintf('Sampling Interval              : %.2f seconds\n',Interval);
fprintf('Active Time per Cycle          : %.2f seconds\n',ActiveTime);
fprintf('Transmission Time per Cycle   : %.2f seconds\n',TransmitTime);
fprintf('Sleep Time per Cycle           : %.2f seconds\n',SleepTime);
fprintf('Average Current                : %.3f mA\n',AverageCurrent);
fprintf('Charge Consumed                 : %.3f mAh\n',TotalCharge);
fprintf('Battery Used                   : %.3f %%\n',BatteryUsed);
fprintf('Estimated Battery Life         : %.2f hours\n',LifeHours);
fprintf('Estimated Battery Life         : %.2f days\n',LifeDays);
fprintf('Estimated Battery Life         : %.2f months\n',LifeMonths);
figure;
plot(LDR,'LineWidth',1.5);
grid on;
xlabel('Sampling Event');
ylabel('LDR Sensor Value');
title('Sensor Data Used for Battery Life Estimation');
figure;
stem(1:N,ones(N,1),'filled');
```

OUTPUT

==========

BATTERY LIFE ESTIMATION RESULTS

==========

Number of Sensor Samples : 68

Number of Sampling Events : 68

Number of Transmission Events : 68

Sampling Interval : 10.00 seconds

Active Time per Cycle : 0.50 seconds

Transmission Time per Cycle : 0.20 seconds

Sleep Time per Cycle : 9.30 seconds

Average Current : 1.559 mA

Charge Consumed : 0.295 mAh

Battery Used : 0.012 %

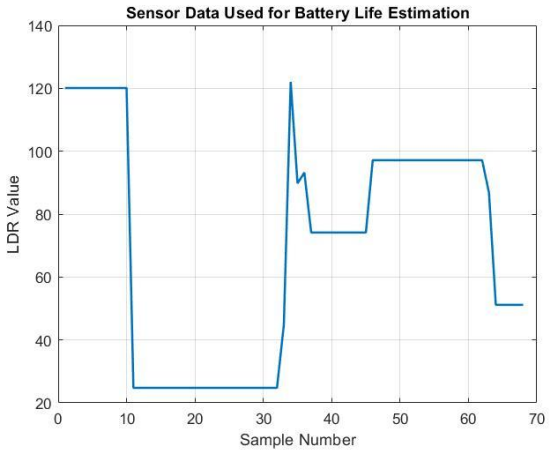
Estimated Battery Life : 1603.28 hours

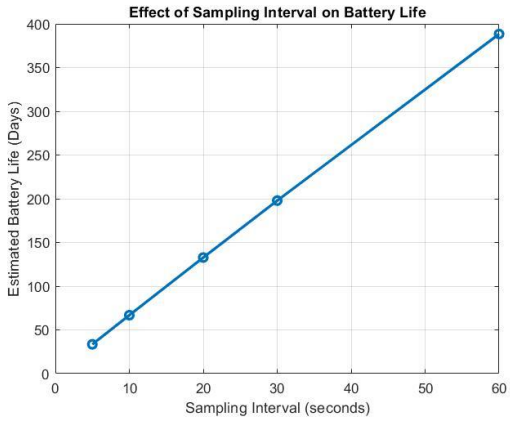
Estimated Battery Life : 66.80 days

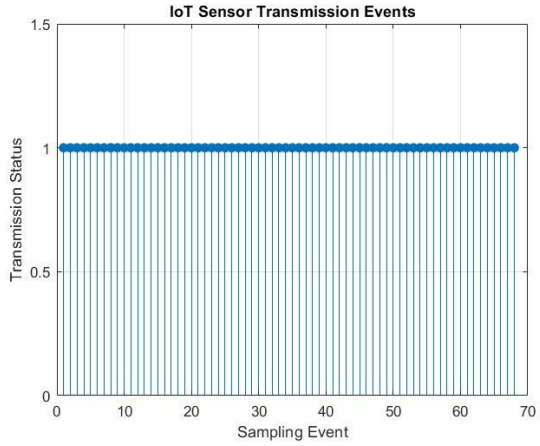
Estimated Battery Life : 2.23 months

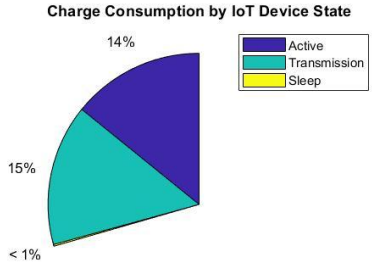
==========

```
grid on;
xlabel('Sampling Event');
ylabel('Transmission Status');
title('IoT Sensor Transmission Events');
ylim([0 1.5]);
Intervals = [5 10 20 30 60];
LifeDays = zeros(size(Intervals));
for i = 1:length(Intervals)
    T = Intervals(i);
    Sleep = T-ActiveTime-TransmitTime;
    Avg = (ActiveCurrent*ActiveTime + ...
          TransmitCurrent*TransmitTime + ...
          SleepCurrent*Sleep)/T;
    LifeDays(i) = BatteryCapacity/Avg/24;
end
figure;
plot(Intervals,LifeDays,'-o','LineWidth',2);
grid on;
xlabel('Sampling Interval (seconds)');
ylabel('Estimated Battery Life (Days)');
title('Effect of Sampling Interval on Battery Life');
figure;
pie([ActiveCharge TransmitCharge SleepCharge]);
title('Charge Consumption by IoT Device State');
legend('Active','Transmission','Sleep');
```


Sensor Data Used for Battery Life Estimation
LDR Value
Sample Number
140
120
100
80
60
40
20
0
10
20
30
40
50
60
70


Effect of Sampling Interval on Battery Life
Estimated Battery Life (Days)
Sampling Interval (seconds)
400
350
300
250
200
150
100
50
0
0
10
20
30
40
50
60


IoT Sensor Transmission Events
Transmission Status
Sampling Event
1.5
1
0.5
0
0
10
20
30
40
50
60
70


Charge Consumption by IoT Device State
14%
15%
< 1%
Active
Transmission
Sleep

Result

The battery life of the IoT device was successfully estimated using MATLAB. The analysis showed that increasing the transmission interval reduced the duty cycle and average current consumption, thereby extending the operational lifetime of the battery-powered IoT device.

Exp No. 8	Edge Data Aggregation in IoT Networks
Date:	

Aim

To implement edge data aggregation techniques for IoT sensor data using MATLAB by summarizing multiple sensor readings before transmission to reduce network traffic and improve communication efficiency.

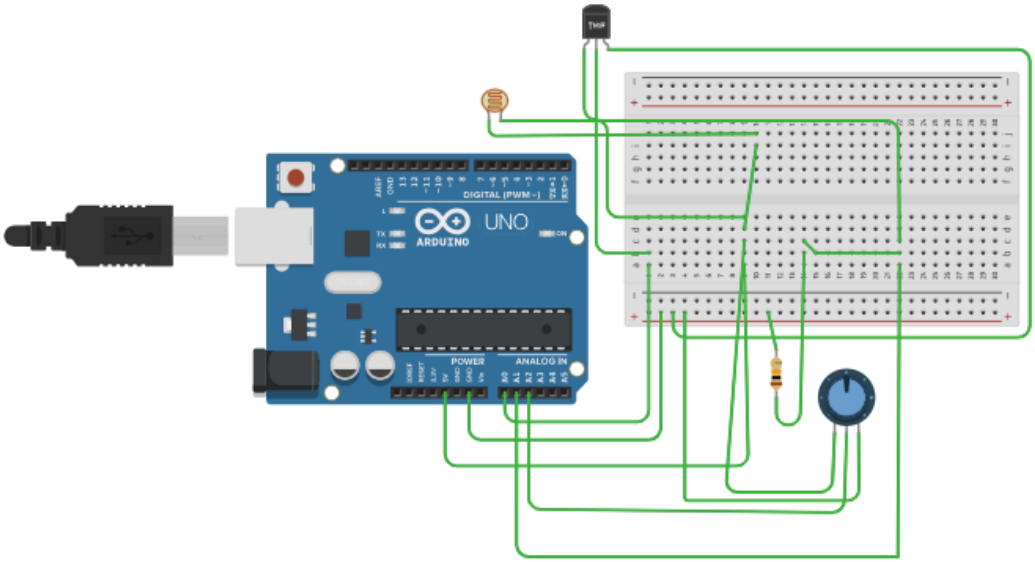
Components Required

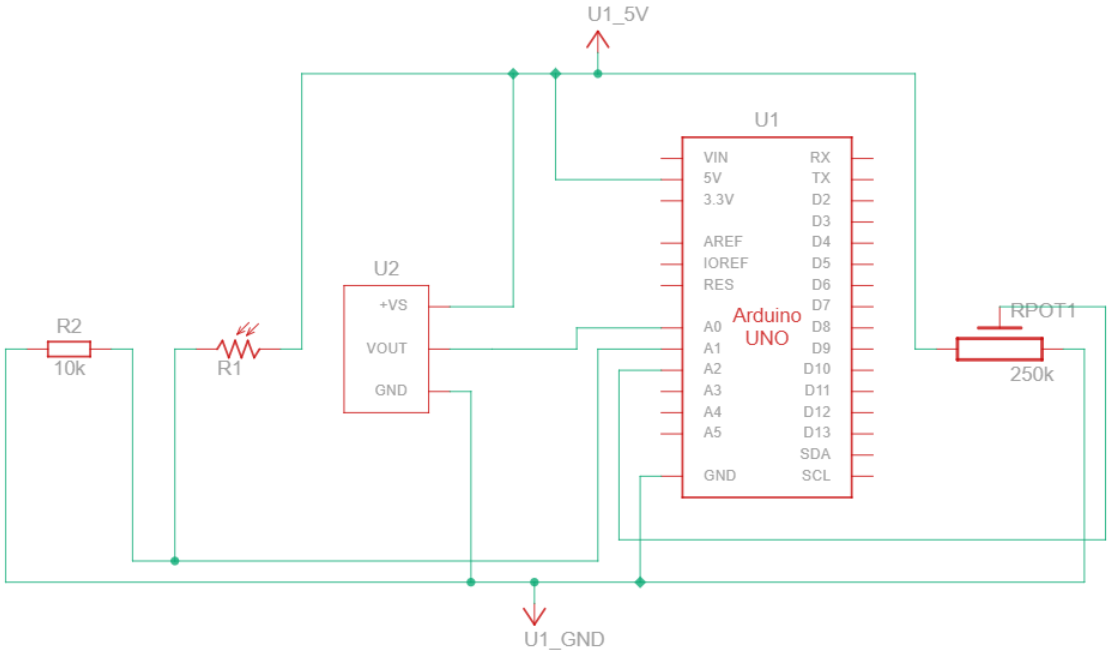
- Arduino UNO
- LDR Sensor
- 10 kΩ Potentiometer
- 10 kΩ Resistor
- Breadboard
- Connecting Wires
- MATLAB
- Tinkercad

Program

```
const int tempPin = A0;
const int lightPin = A1;
const int soilPin = A2;
void setup()
{
  Serial.begin(9600);
}
void loop()
{
  int tempValue = analogRead(tempPin);
  float voltage = tempValue * 5.0 / 1023.0;
  float temperature = (voltage - 0.5) * 100;
  int light = analogRead(lightPin);
```

Diagram


UNO
ARDUINO
DIGITAL (PWM~)
POWER
ANALOG IN


U1_5V
U1
VIN
5V
3.3V
AREF
IOREF
RES
A0
A1
A2
A3
A4
A5
GND
RX
TX
D2
D3
D4
D5
D6
D7
D8
D9
D10
D11
D12
D13
SDA
SCL
Arduino
UNO
U2
+VS
VOUT
GND
R2
10k
R1
RPOT1
250k
U1_GND

```
      int soil = analogRead(soilPin);
      Serial.print(temperature);
      Serial.print(",");
      Serial.print(light);
      Serial.print(",");
      Serial.println(soil);
      delay(1000);
    }
```

MATLAB

```
 clc;
clear;
close all;
 data = csvread('SensorData.csv');
LDR = data(:,1);
POT = data(:,2);
 N = length(LDR);
W = 5;
G = floor(N/W);
 AvgLDR = zeros(G,1);
AvgPOT = zeros(G,1);
MaxLDR = zeros(G,1);
MinLDR = zeros(G,1);
 for i = 1:G
    x = LDR((i-1)*W+1:i*W);
    y = POT((i-1)*W+1:i*W);
    AvgLDR(i) = mean(x);
    AvgPOT(i) = mean(y);
    MaxLDR(i) = max(x);
    MinLDR(i) = min(x);
```

Output

RAW IoT SENSOR DATA

Total Sensor Readings : 68

AGGREGATED DATA AT EDGE NODE

Group	Avg LDR	Avg POT	Max LDR	Min LDR
1	120.09	967.00	120.09	120.09
2	120.09	967.00	120.09	120.09
3	24.78	54.00	24.78	24.78
4	24.78	54.00	24.78	24.78
5	24.78	54.00	24.78	24.78
6	24.78	935.80	24.78	24.78
7	61.24	963.00	122.04	24.78
8	77.95	963.00	93.21	74.14
9	74.14	955.60	74.14	74.14
10	97.12	969.00	97.12	97.12
11	97.12	969.00	97.12	97.12
12	97.12	969.00	97.12	97.12
13	76.69	969.00	97.12	51.17

PACKET REDUCTION ANALYSIS

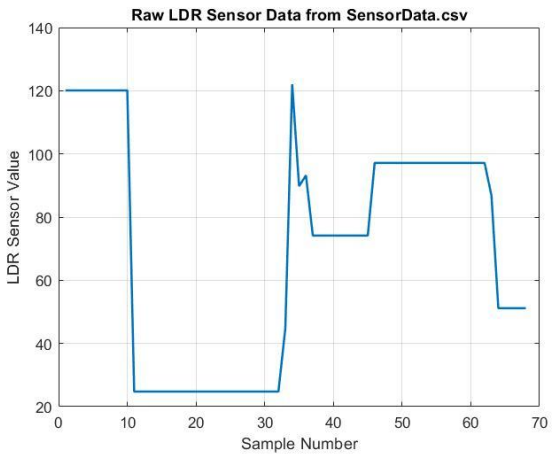
Original Sensor Packets : 68

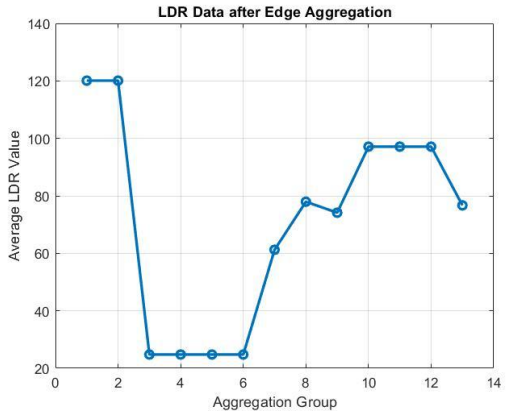
Aggregated Packets : 13

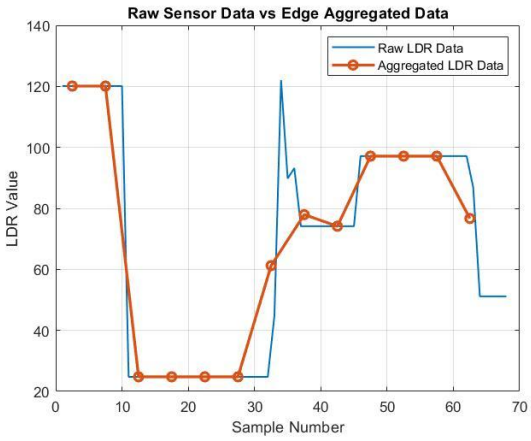
Packet Reduction : 80.88 %

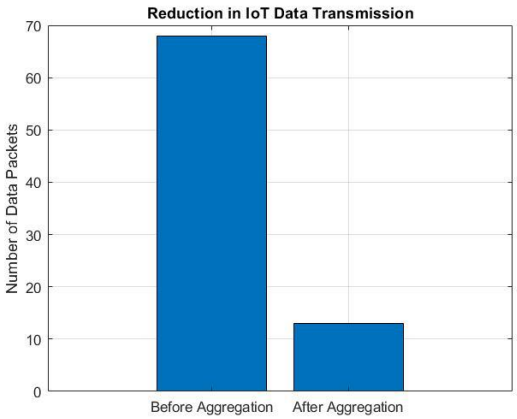
Edge Data Aggregation Completed Successfully

```
end
 OriginalPackets = N;
AggregatedPackets = G;
PacketReduction = (N-G)/N*100;
 figure;
plot(LDR,'LineWidth',1.5);
grid on;
xlabel('Sample Number');
ylabel('LDR Value');
title('Raw LDR Data');
 figure;
plot(AvgLDR,'-o','LineWidth',2);
grid on;
xlabel('Aggregation Group');
ylabel('Average LDR');
title('Aggregated LDR Data');
 figure;
plot(LDR,'LineWidth',1.2);
hold on;
pos = W/2:W:G*W;
plot(pos,AvgLDR,'-o','LineWidth',2);
grid on;
xlabel('Sample Number');
ylabel('LDR Value');
legend('Raw','Aggregated');
 figure;
bar([N G]);
grid on;
set(gca,'XTickLabel',{'Before','After'});
```


Raw LDR Sensor Data from SensorData.csv
LDR Sensor Value
Sample Number
140
120
100
80
60
40
20
0
10
20
30
40
50
60
70


LDR Data after Edge Aggregation
Average LDR Value
Aggregation Group
140
120
100
80
60
40
20
0
2
4
6
8
10
12
14


Raw Sensor Data vs Edge Aggregated Data
Raw LDR Data
Aggregated LDR Data
LDR Value
Sample Number
140
120
100
80
60
40
20
0
10
20
30
40
50
60
70


Reduction in IoT Data Transmission
Number of Data Packets
Before Aggregation
After Aggregation
70
60
50
40
30
20
10
0

```
ylabel('Packets');
title('Packet Reduction');
```

Result

The IoT sensor data was successfully aggregated using MATLAB. Average, maximum, minimum, and event-based aggregation techniques were implemented to summarize sensor readings before transmission. The experiment demonstrated a reduction in the number of transmitted packets, thereby improving communication efficiency and reducing network traffic in IoT systems.

Exp No. 9	IoT Data Annotation with Semantic Metadata Using MATLAB
Date:	

Aim

To implement semantic annotation of IoT sensor data using MATLAB by associating sensor readings with descriptive metadata such as device identification, timestamp, sensor type, location, and measurement unit to improve data interoperability.

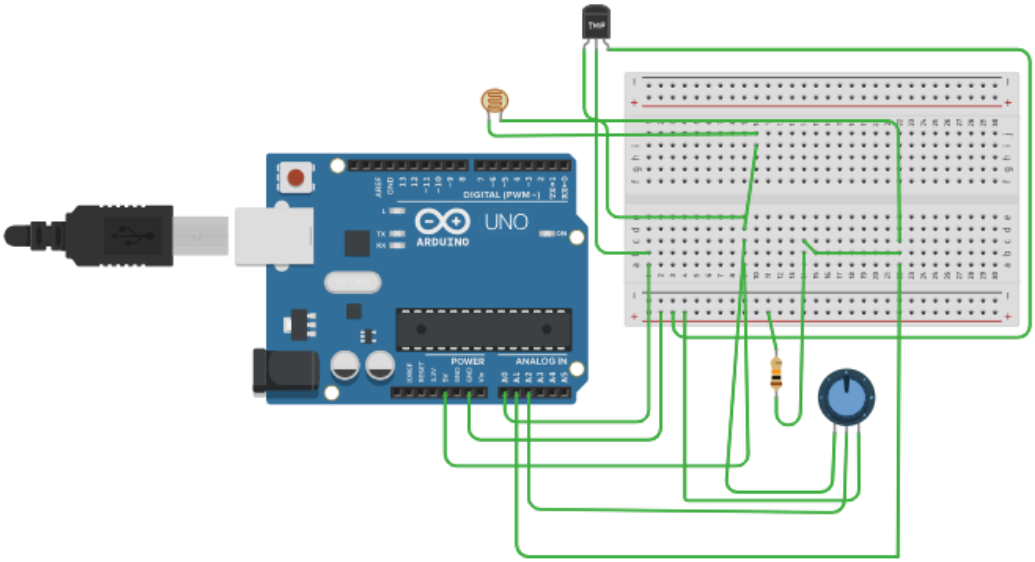
Components Required

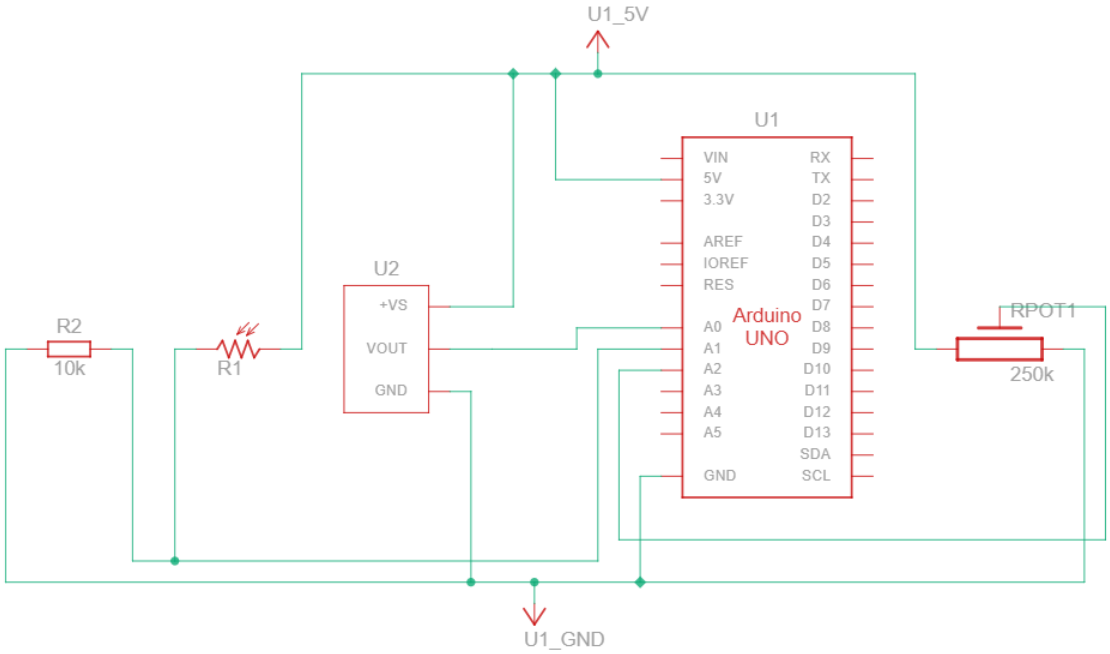
- Arduino UNO
- LDR Sensor
- 10 kΩ Potentiometer
- 10 kΩ Resistor
- Breadboard
- Connecting Wires
- MATLAB
- Tinkercad

Program

```
const int tempPin = A0;
const int lightPin = A1;
const int soilPin = A2;
void setup()
{
  Serial.begin(9600);
}
void loop()
{
  int tempValue = analogRead(tempPin);
  float voltage = tempValue * 5.0 / 1023.0;
  float temperature = (voltage - 0.5) * 100;
  int light = analogRead(lightPin);
```

Diagram


UNO
ARDUINO
DIGITAL (PWM~)
POWER
ANALOG IN


U1_5V
U1
VIN
5V
3.3V
AREF
IOREF
RES
A0
A1
A2
A3
A4
A5
GND
RX
TX
D2
D3
D4
D5
D6
D7
D8
D9
D10
D11
D12
D13
SDA
SCL
Arduino
UNO
U2
+VS
VOUT
GND
R2
10k
R1
RPOT1
250k
U1_GND

```
    int soil = analogRead(soilPin);
    Serial.print(temperature);
    Serial.print(",");
    Serial.print(light);
    Serial.print(",");
    Serial.println(soil);
    delay(1000);
  }
```

MATLAB Code

```
clc;
clear;
close all;
[FileName,FilePath] = uigetfile('*.csv','Select SensorData.csv');
if isequal(FileName,0), return; end
data = csvread(fullfile(FilePath,FileName));
LDR = data(:,1);
POT = data(:,2);
N = length(LDR);
DeviceID = 'NODE001';
Location = 'IoT Laboratory';
Unit = 'Sensor Value';
Timestamp = now+(0:N-1)'/(24*60);
fprintf('Input File : %s\n',FileName);
fprintf('Number of Sensor Samples : %d\n',N);
fprintf('\nLDR Annotated Data\n');
fprintf('DeviceID\tTimestamp\tLocation\tSensor\tUnit\tValue\n');
for i = 1:N
   fprintf('%s\t%.6f\t%s\tLDR Sensor\t%s\t%.2f\n', ...
      DeviceID,Timestamp(i),Location,Unit,LDR(i));
```

Output

```
==========================================
RAW IoT SENSOR DATA
==========================================
Input File          : SensorData.csv
Number of Sensor Samples: 68
==========================================
ANNOTATED LDR DATA
==========================================
   DeviceID     Timestamp       Location          Sensor          Unit         Value
   'NODE001'   7.4024e+05   'IoT Laboratory'   'LDR Sensor'   'Sensor Value'   120.09
   'NODE001'   7.4024e+05   'IoT Laboratory'   'LDR Sensor'   'Sensor Value'   120.09
   'NODE001'   7.4024e+05   'IoT Laboratory'   'LDR Sensor'   'Sensor Value'    24.78
   'NODE001'   7.4024e+05   'IoT Laboratory'   'LDR Sensor'   'Sensor Value'    24.78
   'NODE001'   7.4024e+05   'IoT Laboratory'   'LDR Sensor'   'Sensor Value'    44.82
   'NODE001'   7.4024e+05   'IoT Laboratory'   'LDR Sensor'   'Sensor Value'   122.04
   'NODE001'   7.4024e+05   'IoT Laboratory'   'LDR Sensor'   'Sensor Value'    89.78
   'NODE001'   7.4024e+05   'IoT Laboratory'   'LDR Sensor'   'Sensor Value'    93.21
   'NODE001'   7.4024e+05   'IoT Laboratory'   'LDR Sensor'   'Sensor Value'    74.14
   'NODE001'   7.4024e+05   'IoT Laboratory'   'LDR Sensor'   'Sensor Value'    74.14
   'NODE001'   7.4024e+05   'IoT Laboratory'   'LDR Sensor'   'Sensor Value'    97.12
   'NODE001'   7.4024e+05   'IoT Laboratory'   'LDR Sensor'   'Sensor Value'    97.12
   'NODE001'   7.4024e+05   'IoT Laboratory'   'LDR Sensor'   'Sensor Value'    97.12
   'NODE001'   7.4024e+05   'IoT Laboratory'   'LDR Sensor'   'Sensor Value'    86.85
   'NODE001'   7.4024e+05   'IoT Laboratory'   'LDR Sensor'   'Sensor Value'    51.17
   'NODE001'   7.4024e+05   'IoT Laboratory'   'LDR Sensor'   'Sensor Value'    51.17
==========================================
ANNOTATED POTENTIOMETER DATA
==========================================
```

```
end
fprintf('\nPotentiometer Annotated Data\n');
fprintf('DeviceID\tTimestamp\tLocation\tSensor\tUnit\tValue\n');
for i = 1:N
    fprintf('%s\t%.6f\t%s\tPotentiometer\t%s\t%.2f\n', ...
        DeviceID,Timestamp(i),Location,Unit,POT(i));
end
OutputFolder = uigetdir(FilePath,'Select Folder to Save Annotated Data');
if isequal(OutputFolder,0), return; end
LDRFile = fullfile(OutputFolder,'Annotated_LDR_Data.csv');
POTFile = fullfile(OutputFolder,'Annotated_POT_Data.csv');
fid = fopen(LDRFile,'w');
fprintf(fid,'DeviceID,Timestamp,Location,Sensor,Unit,Value\n');
for i = 1:N
    fprintf(fid,'%s,%.6f,%s,LDR Sensor,%s,%.2f\n', ...
        DeviceID,Timestamp(i),Location,Unit,LDR(i));
end
fclose(fid);
fid = fopen(POTFile,'w');
fprintf(fid,'DeviceID,Timestamp,Location,Sensor,Unit,Value\n');
for i = 1:N
    fprintf(fid,'%s,%.6f,%s,Potentiometer,%s,%.2f\n', ...
        DeviceID,Timestamp(i),Location,Unit,POT(i));
end
fclose(fid);
fprintf('\nAnnotated Data Saved Successfully\n');
fprintf('LDR File : %s\n',LDRFile);
fprintf('POT File : %s\n',POTFile);
figure;
```

DeviceID	Timestamp	Location	Sensor	Unit	Value
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	967
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	967
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	967
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	967
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	54
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	54
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	54
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	54
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	54
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	854
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	962
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	937
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	963
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	963
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	963
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	963
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	963
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	963
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	903
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	968
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	969
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	969
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	969
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	969
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	969
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	969
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	969
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	969

```
plot(LDR,'LineWidth',1.5);
grid on;
xlabel('Sample Number');
ylabel('LDR Sensor Value');
title('Raw LDR Data');
figure;
plot(LDR,'-o','LineWidth',1.5);
grid on;
xlabel('Sample Number');
ylabel('LDR Sensor Value');
title('LDR Data after Semantic Annotation');
```

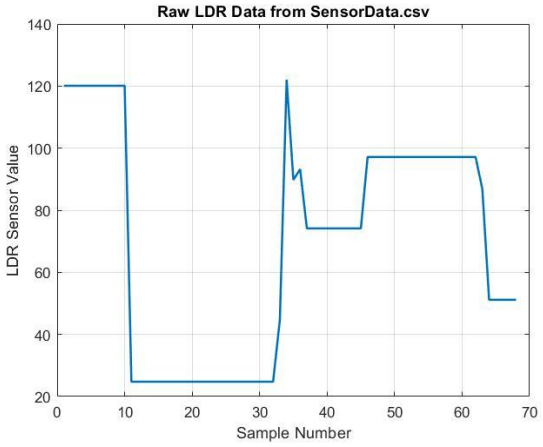
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	969
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	969
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	942
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	942
'NODE001'	7.4024e+05	'IoT Laboratory'	'Potentiometer'	'Sensor Value'	942

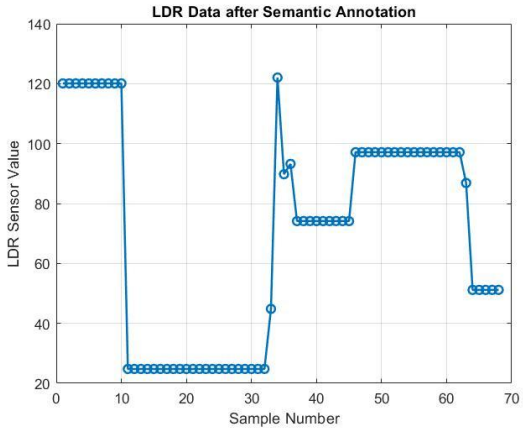
==========

ANNOTATED DATA SAVED SUCCESSFULLY

==========

IoT Data Annotation Completed Successfully.





Result

The IoT sensor data was successfully annotated with semantic metadata using MATLAB. Each sensor reading was enriched with contextual information including device ID, timestamp, location, sensor type, and measurement unit. The annotated dataset improves data interoperability, traceability, and usability for IoT applications and cloud-based analytics.

Exp No. 10	Simulation of Actuator Control in IoT Devices Using MATLAB
Date:	

Aim

To simulate the operation of IoT actuators using MATLAB by controlling devices based on sensor readings acquired from an IoT sensing system.

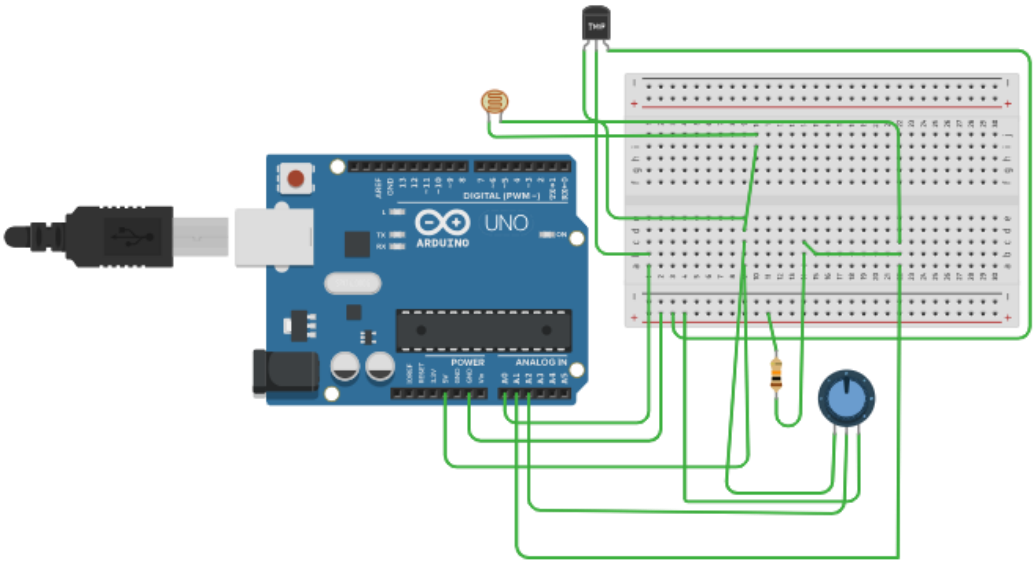
Components Required

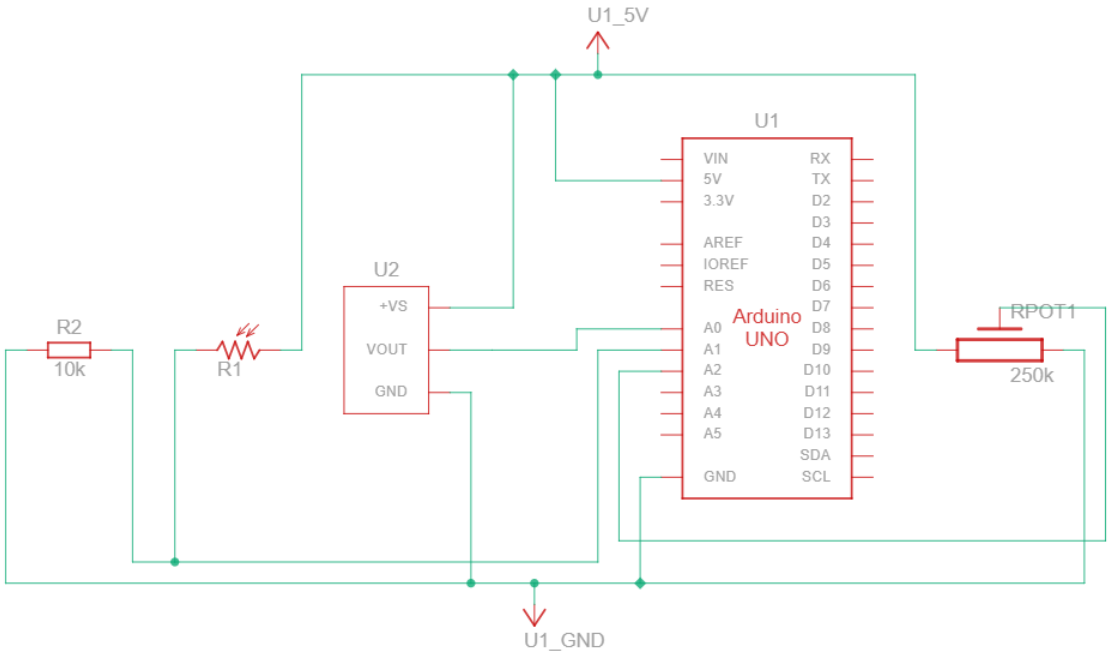
- Arduino UNO
- LDR Sensor
- 10 kΩ Potentiometer
- 10 kΩ Resistor
- Breadboard
- Connecting Wires
- MATLAB
- Tinkercad

Program

```
const int tempPin = A0;
const int lightPin = A1;
const int soilPin = A2;
void setup()
{
  Serial.begin(9600);
}
void loop()
{
  int tempValue = analogRead(tempPin);
  float voltage = tempValue * 5.0 / 1023.0;
  float temperature = (voltage - 0.5) * 100;
  int light = analogRead(lightPin);
```

Diagram


UNO
ARDUINO
DIGITAL (PWM~)
POWER
ANALOG IN


U1_5V
U1
VIN
5V
3.3V
AREF
IOREF
RES
A0
A1
A2
A3
A4
A5
GND
RX
TX
D2
D3
D4
D5
D6
D7
D8
D9
D10
D11
D12
D13
SDA
SCL
Arduino
UNO
U2
+VS
VOUT
GND
R2
10k
R1
RPOT1
250k
U1_GND

```
    int soil = analogRead(soilPin);
    Serial.print(temperature);
    Serial.print(",");
    Serial.print(light);
    Serial.print(",");
    Serial.println(soil);
    delay(1000);
  }
```

MATLAB Code

```
clc;
clear;
close all;
data = csvread('SensorData.csv');
LDR = data(:,1);
POT = data(:,2);
N = length(LDR);
LDRThreshold = 600;
POTThreshold = 350;
LED = zeros(N,1);
Fan = zeros(N,1);
Relay = zeros(N,1);
fprintf('Sample\tLDR\tPOT\tLED\tFAN\tRELAY\n');
for i = 1:N
    LED(i) = LDR(i) < LDRThreshold;
    Fan(i) = POT(i) > POTThreshold;
    Relay(i) = LED(i) && Fan(i);
    fprintf('%d\t%d\t%d\t%d\t%d\t%d\n', ...
        i,LDR(i),POT(i),LED(i),Fan(i),Relay(i));
end
```

Output

```
================================================
      IoT ACTUATOR CONTROL RESULTS
================================================
```

Sample	LDR	POT	LED	FAN	RELAY
1	1.200900e+02		967	1	1	1
2	1.200900e+02		967	1	1	1
3	1.200900e+02		967	1	1	1
4	1.200900e+02		967	1	1	1
5	1.200900e+02		967	1	1	1
6	1.200900e+02		967	1	1	1
7	1.200900e+02		967	1	1	1
8	1.200900e+02		967	1	1	1
9	1.200900e+02		967	1	1	1
10	1.200900e+02		967	1	1	1
11	2.478000e+01		54	1	0	0
12	2.478000e+01		54	1	0	0
13	2.478000e+01		54	1	0	0
14	2.478000e+01		54	1	0	0
15	2.478000e+01		54	1	0	0
16	2.478000e+01		54	1	0	0
17	2.478000e+01		54	1	0	0
18	2.478000e+01		54	1	0	0
19	2.478000e+01		54	1	0	0
20	2.478000e+01		54	1	0	0
21	2.478000e+01		54	1	0	0
22	2.478000e+01		54	1	0	0
23	2.478000e+01		54	1	0	0
24	2.478000e+01		54	1	0	0

```
LED_ON = sum(LED);
Fan_ON = sum(Fan);
Relay_ON = sum(Relay);
fprintf('\nTotal Sensor Samples : %d\n',N);
fprintf('LED Activated        : %d Times\n',LED_ON);
fprintf('Fan Activated        : %d Times\n',Fan_ON);
fprintf('Relay Activated      : %d Times\n',Relay_ON);
figure;
plot(LDR,'LineWidth',1.5);
hold on;
plot([1 N],[LDRThreshold LDRThreshold],'--');
grid on;
xlabel('Sample Number');
ylabel('LDR Value');
title('LDR Sensor Data and Control Threshold');
legend('LDR Data','LDR Threshold');
figure;
plot(POT,'LineWidth',1.5);
hold on;
plot([1 N],[POTThreshold POTThreshold],'--');
grid on;
xlabel('Sample Number');
ylabel('Potentiometer Value');
title('Potentiometer Data and Control Threshold');
legend('POT Data','POT Threshold');
figure;
stairs(LED,'LineWidth',2);
grid on;
xlabel('Sample Number');
```

25	2.478000e+01	54	1	0	0
26	2.478000e+01	854	1	1	1
27	2.478000e+01	962	1	1	1
28	2.478000e+01	937	1	1	1
29	2.478000e+01	963	1	1	1
30	2.478000e+01	963	1	1	1
31	2.478000e+01	963	1	1	1
32	2.478000e+01	963	1	1	1
33	4.482000e+01	963	1	1	1

Total Sensor Samples : 68

LED Activated : 68 Times

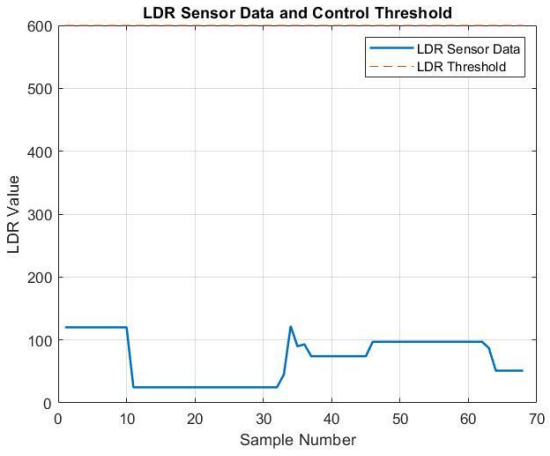
Fan Activated : 53 Times

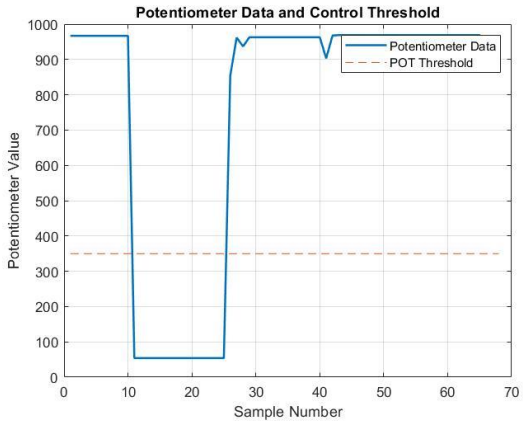
Relay Activated : 53 Times

==========

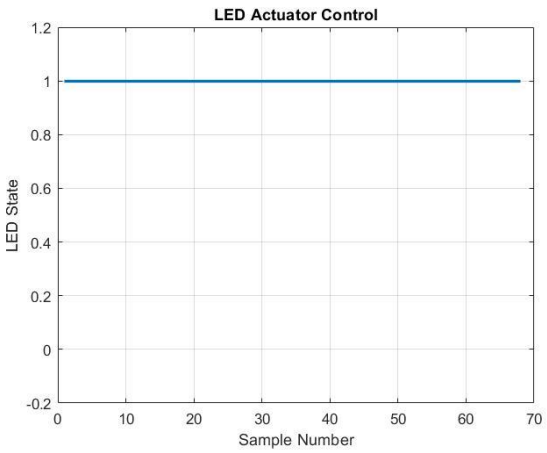
Actuator Control Simulation Completed Successfully

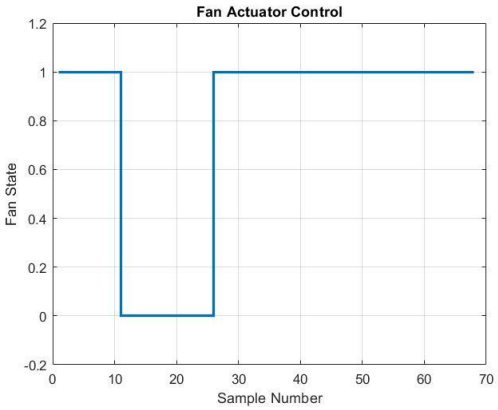
==========

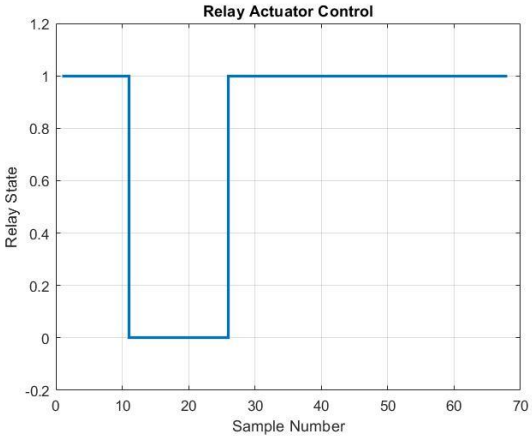


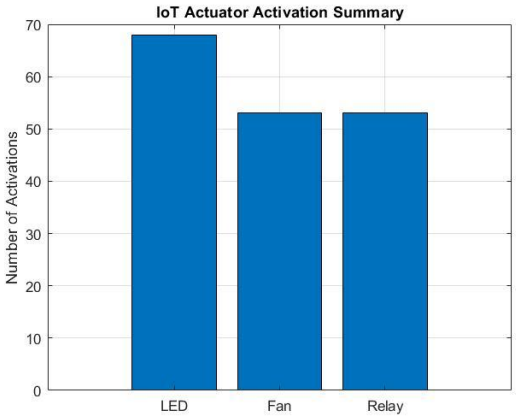


```
ylabel('LED State');
title('LED Actuator Control');
ylim([-0.2 1.2]);
figure;
stairs(Fan,'LineWidth',2);
grid on;
xlabel('Sample Number');
ylabel('Fan State');
title('Fan Actuator Control');
ylim([-0.2 1.2]);
figure;
stairs(Relay,'LineWidth',2);
grid on;
xlabel('Sample Number');
ylabel('Relay State');
title('Relay Actuator Control');
ylim([-0.2 1.2]);
figure;
bar([LED_ON Fan_ON Relay_ON]);
grid on;
set(gca,'XTickLabel',{'LED','Fan','Relay'});
ylabel('Number of Activations');
title('IoT Actuator Activation Summary');
```


LED Actuator Control
LED State
Sample Number
1.2
1
0.8
0.6
0.4
0.2
0
-0.2
0
10
20
30
40
50
60
70


Fan Actuator Control
Fan State
Sample Number
1.2
1
0.8
0.6
0.4
0.2
0
-0.2
0
10
20
30
40
50
60
70


Relay Actuator Control
Relay State
Sample Number
1.2
1
0.8
0.6
0.4
0.2
0
-0.2
0
10
20
30
40
50
60
70


IoT Actuator Activation Summary
Number of Activations
70
60
50
40
30
20
10
0
LED
Fan
Relay

Result

The actuator control logic was successfully simulated using MATLAB. Based on the sensor readings, the LED, fan, and relay were automatically activated or deactivated according to predefined threshold values. The experiment demonstrates how IoT systems can perform automated monitoring and control of physical devices using sensor-based decision making.

Exp No. 11	IoT Device Localization Using Signal Strength
Date:	

Aim

To simulate IoT device localization using BLE signal strength (RSSI) in MATLAB and classify the approximate device location based on the received signal strength.

Components

MATLAB

MATLAB Code

```
clc;
clear;
close all;
N = 50;
pot = zeros(1,N);
RSSI = zeros(1,N);
zone = zeros(1,N);
for i = 1:N
    pot(i) = randi([0 1023]);
    RSSI(i) = -40 - pot(i)/1023*50;
    if RSSI(i) >= -50
        zone(i) = 1;
        location = 'Zone 1 - Very Near';
    elseif RSSI(i) >= -60
        zone(i) = 2;
        location = 'Zone 2 - Near';
    elseif RSSI(i) >= -70
        zone(i) = 3;
        location = 'Zone 3 - Medium';
    elseif RSSI(i) >= -80
        zone(i) = 4;
```

OUTPUT

Sample 1 | Potentiometer = 166 | RSSI = -48.11 dBm | Zone 1 - Very Near

Sample 2 | Potentiometer = 813 | RSSI = -79.74 dBm | Zone 4 - Far

Sample 3 | Potentiometer = 318 | RSSI = -55.54 dBm | Zone 2 - Near

Sample 4 | Potentiometer = 541 | RSSI = -66.44 dBm | Zone 3 - Medium

Sample 5 | Potentiometer = 169 | RSSI = -48.26 dBm | Zone 1 - Very Near

Sample 6 | Potentiometer = 616 | RSSI = -70.11 dBm | Zone 4 - Far

Sample 7 | Potentiometer = 269 | RSSI = -53.15 dBm | Zone 2 - Near

Sample 8 | Potentiometer = 669 | RSSI = -72.70 dBm | Zone 4 - Far

Sample 9 | Potentiometer = 705 | RSSI = -74.46 dBm | Zone 4 - Far

Sample 10 | Potentiometer = 766 | RSSI = -77.44 dBm | Zone 4 - Far

Sample 11 | Potentiometer = 461 | RSSI = -62.53 dBm | Zone 3 - Medium

Sample 12 | Potentiometer =   85 | RSSI = -44.15 dBm | Zone 1 - Very Near

Sample 13 | Potentiometer = 234 | RSSI = -51.44 dBm | Zone 2 - Near

Sample 14 | Potentiometer = 935 | RSSI = -85.70 dBm | Zone 5 - Very Far

Sample 15 | Potentiometer = 156 | RSSI = -47.62 dBm | Zone 1 - Very Near

Sample 16 | Potentiometer = 845 | RSSI = -81.30 dBm | Zone 5 - Very Far

Sample 17 | Potentiometer = 551 | RSSI = -66.93 dBm | Zone 3 - Medium

Sample 18 | Potentiometer = 1020 | RSSI = -89.85 dBm | Zone 5 - Very Far

Sample 19 | Potentiometer =   80 | RSSI = -43.91 dBm | Zone 1 - Very Near

Sample 20 | Potentiometer = 453 | RSSI = -62.14 dBm | Zone 3 - Medium

Sample 21 | Potentiometer = 109 | RSSI = -45.33 dBm | Zone 1 - Very Near

Sample 22 | Potentiometer = 984 | RSSI = -88.09 dBm | Zone 5 - Very Far

Sample 23 | Potentiometer =   4 | RSSI = -40.20 dBm | Zone 1 - Very Near

Sample 24 | Potentiometer = 793 | RSSI = -78.76 dBm | Zone 4 - Far

Sample 25 | Potentiometer = 836 | RSSI = -80.86 dBm | Zone 5 - Very Far

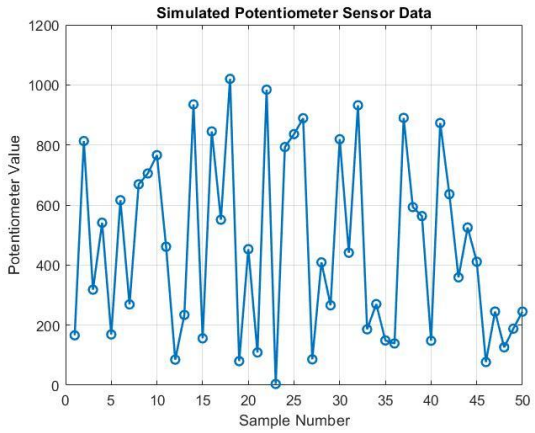
Sample 26 | Potentiometer = 889 | RSSI = -83.45 dBm | Zone 5 - Very Far

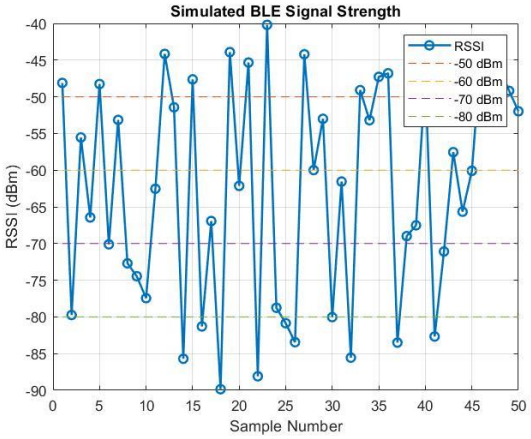
Sample 27 | Potentiometer =   86 | RSSI = -44.20 dBm | Zone 1 - Very Near

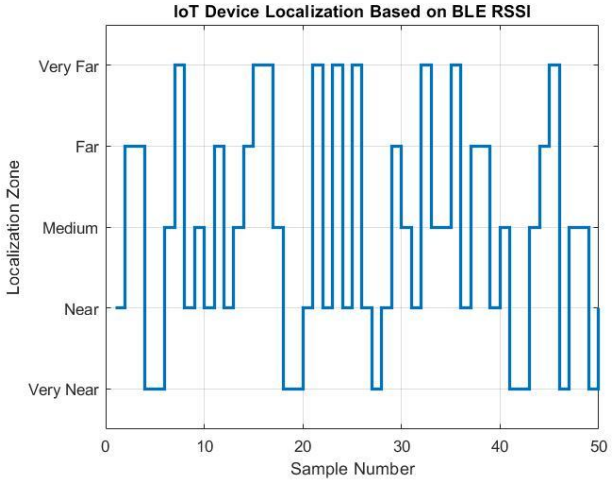
Sample 28 | Potentiometer = 409 | RSSI = -59.99 dBm | Zone 2 - Near

```
        location = 'Zone 4 - Far';
    else
        zone(i) = 5;
        location = 'Zone 5 - Very Far';
    end
    fprintf('Sample %2d | Potentiometer = %4d | RSSI = %6.2f dBm | %s\n', ...
        i,pot(i),RSSI(i),location);

    pause(0.2);
end
figure;
plot(pot,'o-','LineWidth',1.5);
grid on;
xlabel('Sample Number');
ylabel('Potentiometer Value');
title('Simulated Potentiometer Sensor Data');
figure;
plot(RSSI,'o-','LineWidth',1.5);
hold on;
plot([1 N],[-50 -50],'--');
plot([1 N],[-60 -60],'--');
plot([1 N],[-70 -70],'--');
plot([1 N],[-80 -80],'--');
grid on;
xlabel('Sample Number');
ylabel('RSSI (dBm)');
title('Simulated BLE Signal Strength');
figure;
stairs(zone,'LineWidth',2);
```


Simulated Potentiometer Sensor Data
Potentiometer Value
Sample Number
0
200
400
600
800
1000
1200
0
5
10
15
20
25
30
35
40
45
50


Simulated BLE Signal Strength
RSSI (dBm)
Sample Number
RSSI
-50 dBm
-60 dBm
-70 dBm
-80 dBm
-40
-45
-50
-55
-60
-65
-70
-75
-80
-85
-90
0
5
10
15
20
25
30
35
40
45
50


IoT Device Localization Based on BLE RSSI
Localization Zone
Very Far
Far
Medium
Near
Very Near
Sample Number
0
10
20
30
40
50

```
grid on;
xlabel('Sample Number');
ylabel('Localization Zone');
title('IoT Device Localization Based on BLE RSSI');
set(gca,'YTick',1:5);
set(gca,'YTickLabel',{'Very Near','Near','Medium','Far','Very Far'});
ylim([0.5 5.5]);
```

Result

The IoT device localization system was successfully simulated using MATLAB. The simulated RSSI variation and corresponding localization zones were successfully displayed using MATLAB plots.

Exp No. 12 Date:	Firmware Update Management in IoT Devices Using MATLAB

Aim

To simulate firmware update management for IoT devices using MATLAB by checking firmware versions, validating update integrity, and managing the firmware installation process.

Components

MATLAB

MATLAB Code

```
clc;
clear;
close all;
CurrentVersion = 1.2;
LatestVersion = 1.5;
fprintf('Current Firmware Version : %.1f\n',CurrentVersion);
fprintf('Latest Firmware Version  : %.1f\n',LatestVersion);
if LatestVersion > CurrentVersion
    disp('New Firmware Available');
    disp('Checking Update...');
    pause(1);
    disp('Downloading Firmware...');
    pause(2);
    DownloadStatus = 1;
    ChecksumServer = 54893;
    ChecksumDownloaded = 54893;
    if DownloadStatus && ChecksumServer == ChecksumDownloaded
        disp('Firmware Integrity Verified');
        pause(1);
        disp('Installing Firmware...');
```

```
        pause(2);
        CurrentVersion = LatestVersion;
        disp('Restarting IoT Device...');
        pause(2);
        disp('Firmware Update Successful');
    else
        disp('Firmware Verification Failed');
    end
else
    disp('Device Already Updated');
end
fprintf('Current Firmware Version : %.1f\n',CurrentVersion);
Stages = {'Check','Download','Verify','Install','Restart','Complete'};
Progress = [10 30 50 70 90 100];
figure;
plot(Progress,'-o','LineWidth',2);
grid on;
set(gca,'XTick',1:6,'XTickLabel',Stages);
xlabel('Firmware Update Stages');
ylabel('Completion (%)');
title('Firmware Update Progress');
figure;
pie([1 5],{'Updated','Remaining Tasks'});
title('Firmware Update Status');
```

OUTPUT

Firmware Update Management

Current Firmware Version : 1.2

Latest Firmware Version : 1.5

New Firmware Available

Checking Update...

Downloading Firmware...

Firmware Integrity Verified

Installing Firmware...

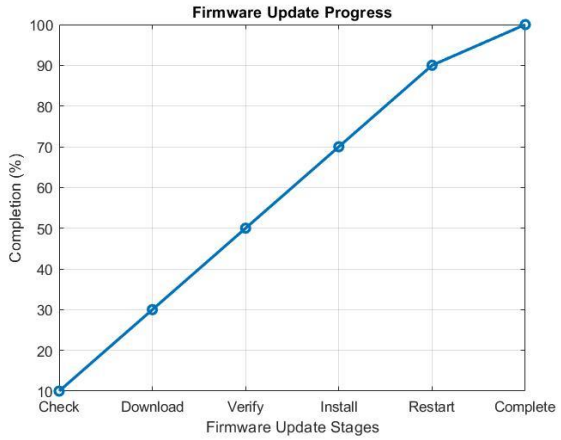
Restarting IoT Device...

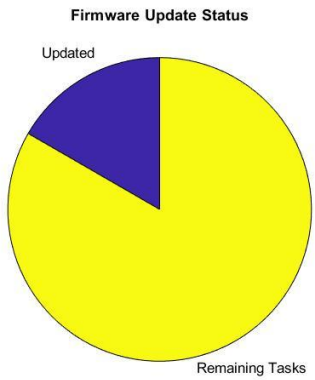
Firmware Update Successful

Current Firmware Version : 1.5

Firmware Update Management Completed Successfully

OUTPUT





Result

The firmware update management process was successfully simulated using MATLAB. It demonstrates the importance of secure firmware management in maintaining reliable and secure IoT devices.

Exp No. 13	Smart Irrigation Control Using Soil Moisture and Temperature
Date:	

Aim:

To simulate a smart irrigation controller that switches a water pump ON or OFF using soil moisture and temperature readings, and to compare its water usage and soil condition with a fixed-timer irrigation system using MATLAB.

Components Required:

- Arduino Uno
- Breadboard
- TMP36 Temperature Sensor
- 10 k potentiometer (to represent the soil moisture sensor)
- LED (to represent the water pump)
- 220 ohm resistor
- Jumper wires
- USB cable
- MATLAB software

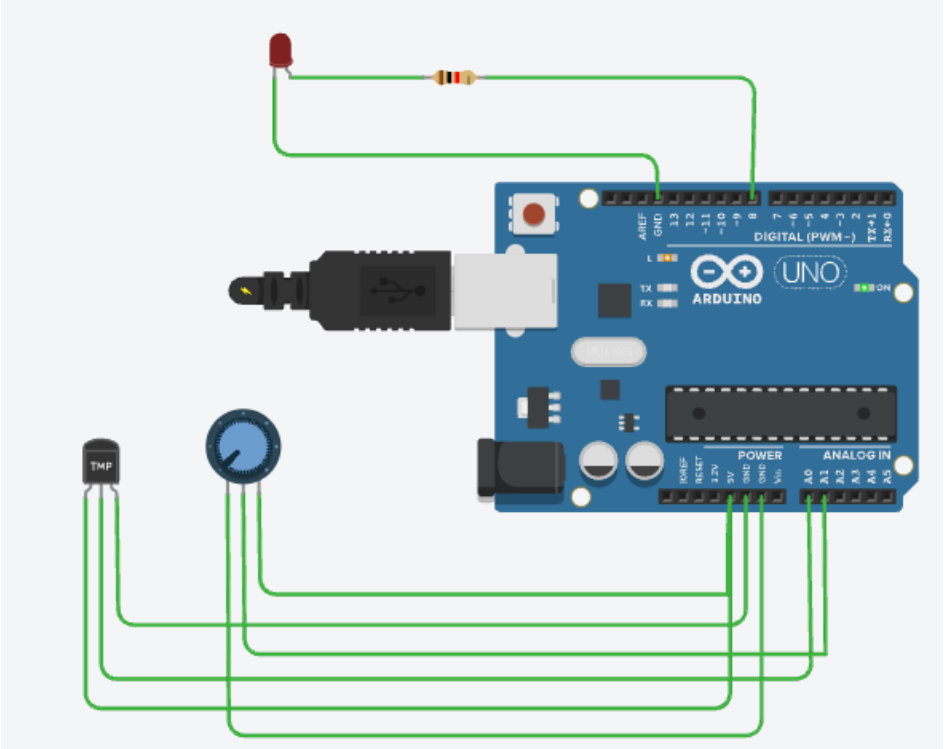
Theory:

Agriculture uses most of the fresh water consumed in the world, and a large part of it is wasted by over-irrigation or by irrigating at the wrong time. Smart irrigation is an IoT application in which sensors measure the condition of the soil and the weather, a controller processes the data, and an actuator (pump or valve) supplies water only when it is needed. This saves water and energy and keeps the crop healthy.

- Sensing layer: soil moisture and temperature sensors measure the field conditions.
- Edge processing layer: the microcontroller (Arduino) reads the sensors and takes the pump decision locally, so the system keeps working even when the network is down. This is the edge decision making studied in Unit I.
- Actuation layer: a relay or MOSFET driven pump or solenoid valve. In this experiment an LED represents the pump.
- Communication and cloud layer: the sensor data and pump status can be sent through LoRa, ZigBee or Wi-Fi (Unit II) using protocols such as MQTT (Unit III) for monitoring and record keeping.

Soil moisture sensor: resistive sensors measure the resistance between two probes, and capacitive sensors measure the change in capacitance of the soil. Both give an analog voltage that is read by the 10-bit ADC of the Arduino (values 0 to 1023). In this experiment a potentiometer represents the sensor, and the reading is converted to a percentage, Soil (%) = ADC value x 100 / 1023.

Diagram:



Component	Pin	Connected To
TMP36	Left / Vout / Right	5V / Arduino A0 / GND
Potentiometer (soil moisture)	Left / Wiper / Right	GND / Arduino A1 / 5V
LED (pump indicator)	Anode / Cathode	D8 through 220 ohm resistor / GND
Arduino Uno	USB	PC (Serial Monitor at 9600 baud)

Program:

```
const int tempPin = A0;
const int soilPin = A1;
const int pumpPin = 8;

void setup() {
  pinMode(pumpPin, OUTPUT);
  Serial.begin(9600);
  Serial.println("Time(s),Temperature(C),Soil(%),Pump");
}
void loop() {
  float voltage = analogRead(tempPin) * (5.0 / 1023.0);
  float tempC = (voltage - 0.5) * 100.0;
  int soil = map(analogRead(soilPin), 0, 1023, 0, 100);
  int pump = 0;

  if (soil < 30) {
    pump = 1;                    // Dry soil
  } else if (soil < 45 && tempC > 32) {
    pump = 1;                    // Drying fast in hot weather
  }
  digitalWrite(pumpPin, pump);

  Serial.print(millis() / 1000.0);
  Serial.print(",");
  Serial.print(tempC);
  Serial.print(",");
  Serial.print(soil);
  Serial.print(",");
  Serial.println(pump);
  delay(1000);
}
```

Procedure:

1. Open Tinkercad, sign in and create a new circuit.
2. Place an Arduino Uno, a breadboard, a TMP36 temperature sensor, a potentiometer, an LED and a 220 ohm resistor on the workspace.
3. Connect the components as shown in the circuit diagram: TMP36 Vout to A0, potentiometer wiper to A1, and the LED (through the 220 ohm resistor) to pin D8 and GND. Connect the 5V and GND pins of the TMP36 and the potentiometer.

4. Click Code, select Text, delete the default code and type the Arduino program given above.
5. Click Start Simulation and open the Serial Monitor.
6. Click the TMP36 and set its temperature (for example 25 C, then 35 C). Rotate the potentiometer to change the soil moisture value.
7. Check the four rules. The LED (pump) must turn ON when soil moisture is below 30 %, or when it is between 30 % and 45 % and the temperature is above 32 C, and OFF otherwise. Note the readings in an observation table.
8. Copy the serial output (Time, Temperature, Soil, Pump) for the record.
9. Open MATLAB, create a new script, type the MATLAB program given below and save it as smart_irrigation.m.
10. Click Run. The program simulates 48 hours of operation, so it needs no input file.
11. Observe the pump events and the smart control versus fixed timer comparison in the Command Window.
12. Observe the four figures: temperature profile, soil moisture, pump operation and water consumption.
13. Compare the water used and the number of dry hours for the two methods, and write the result.

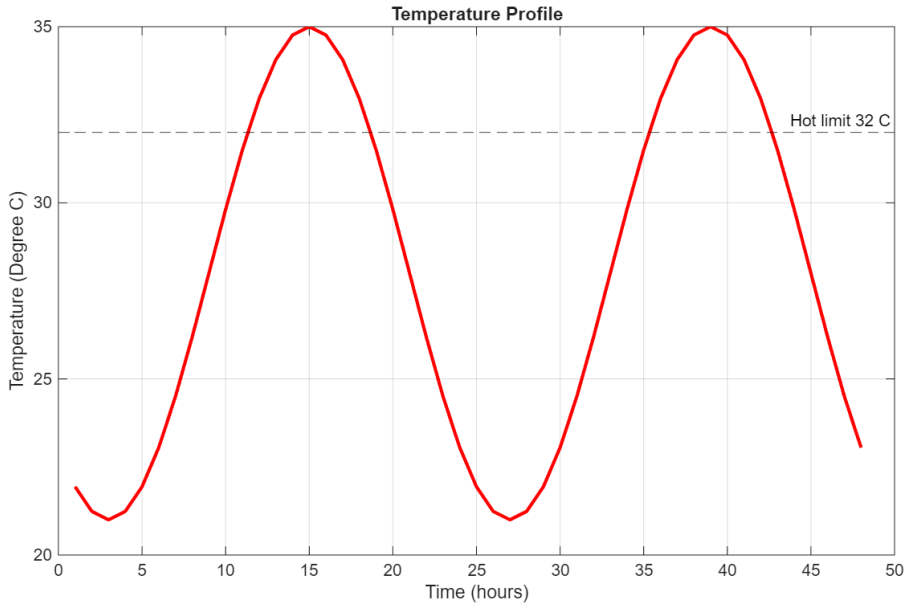
MATLAB Program

```
clc;
clear;
close all;
%% Simulation Settings
Hours = 48;
k = 1:Hours;
Temp = 28 + 7*sin(2*pi*(k-9)/24);   % Temperature profile (Degree C)
Rate = 2;                           % Soil moisture gain (%) per minute of watering
Flow = 2;                           % Pump flow (litre per minute)
%% Smart Irrigation Control (Rule Based)
Soil = zeros(1,Hours+1);
Soil(1) = 55;
Pump = zeros(1,Hours);
disp('------------------------------------------------------------');
disp('        Smart Irrigation Control - Pump Events');
disp('------------------------------------------------------------');
for i = 1:Hours
   Evap = 0.15*(Temp(i) - 18);       % Evaporation loss (%) per hour
   if Soil(i) < 30                   % Dry soil
      if Temp(i) > 32
         Pump(i) = 10;
      else
```

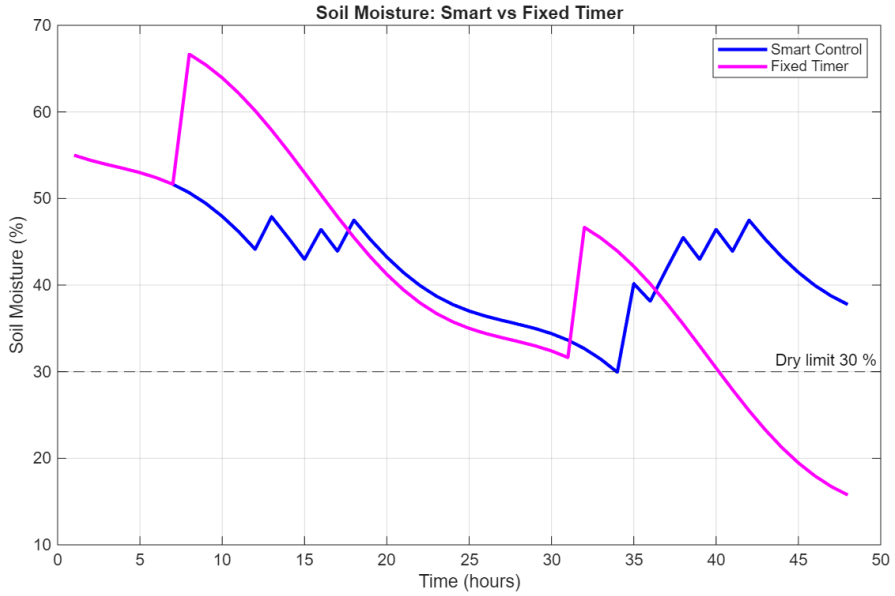
Output:

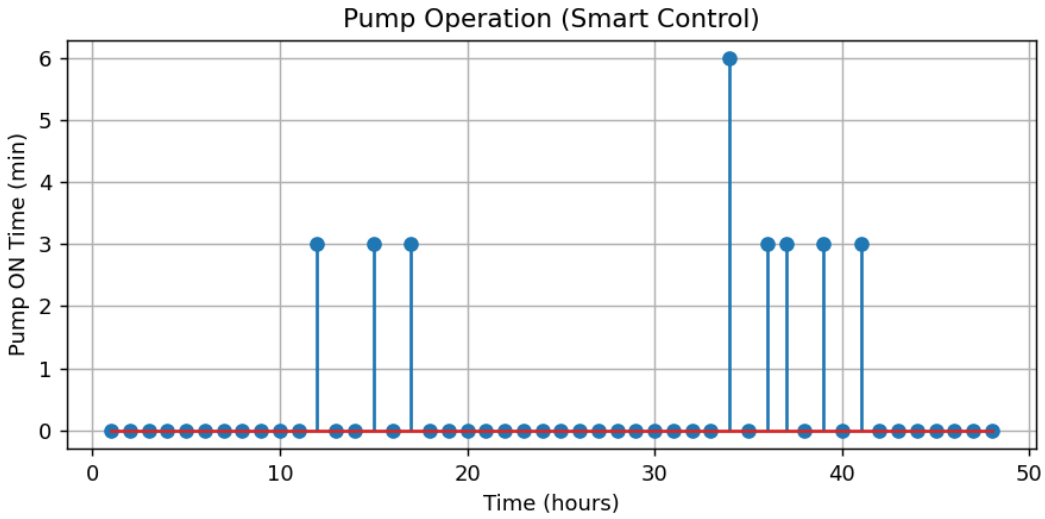
Command Window

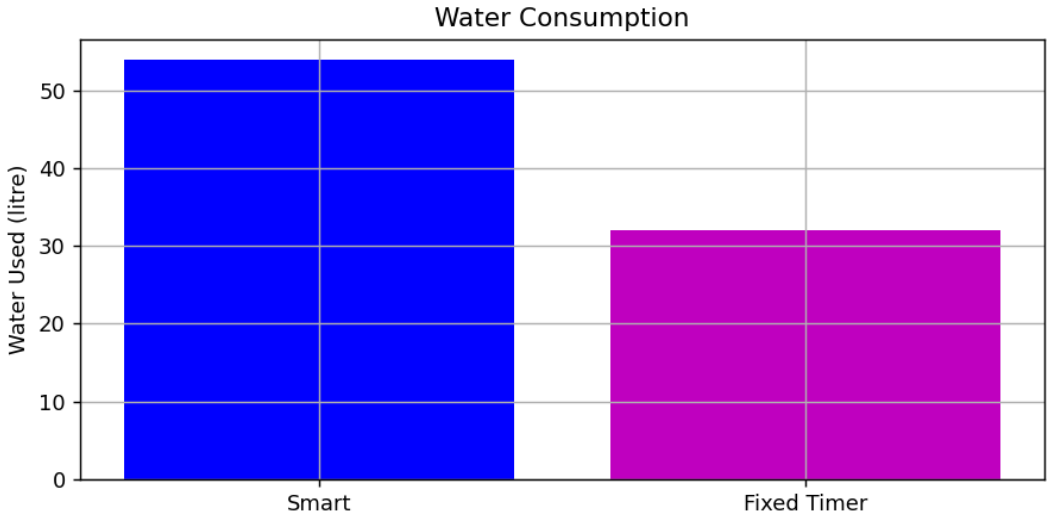
```
------------------------------------------------------------
        Smart Irrigation Control - Pump Events
------------------------------------------------------------
Hour 12 : Soil =  44.1 %  Temp = 32.9 C  -> Pump ON for  3 min
Hour 15 : Soil =  43.0 %  Temp = 35.0 C  -> Pump ON for  3 min
Hour 17 : Soil =  43.9 %  Temp = 34.1 C  -> Pump ON for  3 min
Hour 34 : Soil =  29.9 %  Temp = 29.8 C  -> Pump ON for  6 min
Hour 36 : Soil =  38.1 %  Temp = 32.9 C  -> Pump ON for  3 min
Hour 37 : Soil =  41.9 %  Temp = 34.1 C  -> Pump ON for  3 min
Hour 39 : Soil =  43.0 %  Temp = 35.0 C  -> Pump ON for  3 min
Hour 41 : Soil =  43.9 %  Temp = 34.1 C  -> Pump ON for  3 min
------------------------------------------------------------
Comparison for 48 Hours
                          Smart      Fixed Timer
Pump ON Events        :     8          2
Total Pump Time (min) :    27         16
Water Used (litre)    :    54         32
Hours with Dry Soil   :     1          8
Minimum Soil Moisture :  29.9 %     15.8 %
```



```
            Pump(i) = 6;
        end
    elseif Soil(i) < 45 && Temp(i) > 32   % Drying fast in heat
        Pump(i) = 3;
    end
    if Pump(i) > 0
        fprintf('Hour %2d : Soil = %5.1f %%  Temp = %4.1f C  -> Pump ON for %2d
min\n', ...
            i,Soil(i),Temp(i),Pump(i));
    end
    Soil(i+1) = min(100,max(0,Soil(i) - Evap + Rate*Pump(i)));
end
%% Fixed Timer Irrigation (for comparison)
SoilF = zeros(1,Hours+1);
SoilF(1) = 55;
PumpF = zeros(1,Hours);
for i = 1:Hours
    Evap = 0.15*(Temp(i) - 18);
    if mod(i,24) == 7               % 8 minutes at 7 AM every day
        PumpF(i) = 8;
figure;
stem(k,Pump,'filled','LineWidth',1.5);
grid on;
xlabel('Time (hours)');
ylabel('Pump ON Time (min)');
title('Pump Operation (Smart Control)');
figure;
bar([WaterSmart WaterFixed]);
grid on;
ylabel('Water Used (litre)');
title('Water Consumption');
end
    SoilF(i+1) = min(100,max(0,SoilF(i) - Evap + Rate*PumpF(i)));
end
%% Performance Comparison
WaterSmart = sum(Pump)*Flow;
WaterFixed = sum(PumpF)*Flow;
DrySmart = sum(Soil(1:Hours) < 30);
DryFixed = sum(SoilF(1:Hours) < 30);
disp('Comparison for 48 Hours');
fprintf('Pump ON Events        : %5d      %5d\n',sum(Pump>0),sum(PumpF>0));
fprintf('Total Pump Time (min) : %5d      %5d\n',sum(Pump),sum(PumpF));
fprintf('Water Used (litre)    : %5d      %5d\n',WaterSmart,WaterFixed);
fprintf('Hours with Dry Soil   : %5d      %5d\n',DrySmart,DryFixed);
```


Soil Moisture: Smart vs Fixed Timer
Smart Control
Fixed Timer
Dry limit 30 %
Soil Moisture (%)
Time (hours)
70
60
50
40
30
20
10
0
5
10
15
20
25
30
35
40
45
50


Pump Operation (Smart Control)
Pump ON Time (min)
Time (hours)
6
5
4
3
2
1
0
0
10
20
30
40
50


Water Consumption
Water Used (litre)
50
40
30
20
10
0
Smart
Fixed Timer

```
fprintf('Minimum Soil Moisture : %5.1f %%    %5.1f
%%\n',min(Soil(1:Hours)),min(SoilF(1:Hours)));
figure;
plot(k,Temp,'r','LineWidth',2);
hold on;
yline(32,'k--','Hot limit 32 C');
grid on;
xlabel('Time (hours)');
ylabel('Temperature (Degree C)');
title('Temperature Profile');
figure;
plot(k,Soil(1:Hours),'b','LineWidth',2);
hold on;
plot(k,SoilF(1:Hours),'m','LineWidth',2);
yline(30,'k--','Dry limit 30 %');
grid on;
xlabel('Time (hours)');
ylabel('Soil Moisture (%)');
title('Soil Moisture: Smart vs Fixed Timer');
legend('Smart Control','Fixed Timer');
```

Result:

A smart irrigation controller was simulated using soil moisture and temperature rules in MATLAB. Over 48 hours, the smart controller kept the soil moisture at or above about 30 % (dry soil for 1 hour only) by using 54 litres of water, whereas the fixed timer used 32 litres but left the soil dry for 8 hours, with moisture falling to 15.8 %. Hence the smart controller protects the crop by watering only when the soil and temperature require it.